

DEVELOPMENT STORY

Building a Next-Generation Recommendation System with AI Agent Teams

One Desktop GPU, a Team of Three, and the Record of AI Collaboration

*“AI writes the code,
but design decisions remain with humans.”*

This document records the journey of building a 13-task,
7-expert
PLE+adaTT recommendation system — without
infrastructure budget,
using a single desktop GPU and a team of AI agents.

AI Ops PLE Platform

Project Background

Team Composition and Constraints

The project team consisted of three people: one data scientist serving as PM/Lead Architect and two engineers. There was no dedicated infrastructure budget, and the only GPU available for development was a single desktop-grade RTX 4070 (12GB VRAM) installed in a local PC.

Constraint Summary

- **Team size:** 3 members (PM/Lead Architect/Data Scientist 1 + Engineers 2)
- **Infrastructure:** No dedicated GPU server. One RTX 4070 12GB (local)
- **Budget:** No infrastructure procurement budget. AWS SageMaker spot instances used
- **Objective:** Build a next-generation system to replace the existing ALS-based recommendation system

The Reality of Infrastructure Constraints

Support from the organization was virtually nonexistent. Requests for GPU resources were met with “there is nothing we can do,” and all project-related expenses — AI tool subscriptions (Claude Code, Gemini, Cursor), AWS cloud costs (SageMaker Spot instances, S3 storage), peripherals, and meals — were covered personally by the PM/Lead Architect.

The data collection environment was equally challenging. Requests to migrate to Spark or Impala were denied, forcing the team to work within a HIVE-based bottleneck environment. To overcome this, parallel query logic was designed from scratch to maximize network bandwidth utilization.

The workspace was an unventilated area adjacent to the server room, without adequate cooling.

The two team members were not formally contracted employees but youth advisory supporters — recent graduates participating in the project while preparing for employment.

All of these constraints ultimately gave the design its identity.

Just as selection pressure in biological evolution drives species specialization, resource constraints acted as selection pressure on the architecture. The 12GB VRAM ceiling fundamentally blocked the lazy approach of “scale parameters for performance” and instead forced evolution toward “encode expressiveness through structural inductive biases.” Seven heterogeneous experts each encoding a distinct mathematical perspective, feature engineering borrowing structural isomorphisms from eleven academic disciplines, lightweight expert designs that run in FP32 on a single desktop GPU — all of these are adaptations born from constraint.

Had a large GPU cluster been available, this architecture would likely never have been conceived. The team would have stacked seven Transformer-based heavy experts and brute-forced the problem with parameters — a forgettable approach. Without constraint, there would have been no identity.

On-Premises System Scale

The on-prem system was not a prototype but a production-scale system: 80+ Airflow DAGs, Champion-Challenger model competition, weekly automated retraining, 734D feature tensor, 18 simultaneous tasks (13 in the AWS benchmark version after removing 5 deterministic-leakage/redundant tasks), and 62 data table ingestion. Building a system of this scale with 3 people was itself a result of AI-augmented development.

Project Objective

The existing financial product recommendation system was based on ALS (Alternating Least Squares) collaborative filtering. The goal was to replace it with a multi-task deep learning recommendation system built on PLE (Progressive Layered Extraction) + adaTT (Adaptive Task Transfer) architecture. The system processes 13 tasks through 7 expert networks, explicitly modeling inter-task relationships.

Architecture Decision Journey

Arriving at the final architecture required exploring and rejecting several candidates.

The legacy system was ALS-based collaborative filtering. The first alternative considered was the **Black-Litterman model**. Originating from portfolio theory, this model combines expert views with market equilibrium via Bayesian updating. However, it was impossible to decompose “how much each model contributed” from the blended posterior distribution. In financial practice, recommendations must be explained to customers, branch staff, and regulators alike — Bayesian updating rendered individual model contributions opaque, making business explanation impossible.

The second alternative was a **model ensemble** — training N independent models and combining their outputs. This introduced N management points, N times the serving cost, and explanations like “MLP #3 contributed 28%” were meaningless in a business context. Both cost and explainability were problematic.

A critical reframing emerged from this process: rather than combining experts **outside** a model, what if we combined them **inside** a single model? This was the turning point toward PLE’s heterogeneous expert architecture.

Task group design also involved trial and error. An initial attempt with **GMM cluster subheads** — $K \text{ clusters} \times T \text{ tasks} = K \times T$ complexity explosion — was abandoned in favor of 4 Financial DNA task groups (product holding probability, next product, customer value, churn risk), stabilizing the structure.

Organizing AI Agents

Phase-by-Phase AI Tool Deployment

Different AI tools were strategically deployed at each stage of the project. The key was leveraging each tool's strengths through deliberate role assignment.

Phase A: Ideation (Gemini)

Gemini was used for initial concept exploration and brainstorming. Its broad knowledge base proved effective for rapidly scanning architecture candidates and comparing the pros and cons of various approaches. ALS replacement options, Black-Litterman exploration, and model ensemble approaches were all explored through dialogue with Gemini.

The greatest value emerged from cross-disciplinary feature ideation. Questions like “Can chemical kinetics describe spending behavior?” and “Is product adoption structurally equivalent to an epidemic?” were posed to systematically identify academic domains with structurally isomorphic problems to financial customer behavior. The PM/Lead Architect brought domain expertise (FRM certification, credit analysis experience) while Gemini provided cross-domain connections. Through this process, the concept of “structural isomorphism” naturally emerged as a central design principle.

Gemini's broad knowledge was optimal not for depth in any single technology, but for rapidly scanning “which field has already solved a similar problem.” The design direction of importing features from 11 academic disciplines was established during this phase and became the foundation for all subsequent technical decisions.

Phase B: Technical Validation (Claude Opus)

Claude Opus was employed to develop ideation results into concrete architectures. It was focused on tasks requiring technical depth, such as loss function design demanding mathematical rigor, data leakage verification, and normalization pipeline design.

Each expert's feasibility was validated one by one: “Can HGCN work on MCC hierarchy?”, “Is Mamba efficient enough for 17-month sequences?” Deep technical discussions with Opus covered PLE vs MMoE trade-offs, adaTT loss-level vs representation-level transfer comparisons, and other architecture-level analyses. Notably, the expert collapse problem in homogeneous MoE was first identified through conversation with Opus, which later led to the discovery of the NeurIPS 2024 sigmoid gate paper.

Opus was also effective at challenging assumptions. Its counterargument — “Is Black-Litterman really suitable?” — accelerated the pivot toward PLE. During this phase, 19 technical reference documents (.typ files) were co-authored with Opus, serving as design specifications that each agent would reference during the subsequent implementation phase.

Phase C: Code Environment Setup (Cursor)

GitHub-based code environment configuration, project structure design, and initial boilerplate generation were performed with Cursor. Its strength lay in fast code navigation and refactoring within an integrated IDE environment.

During this phase, 6 initial design documents (00-09 architecture specifications) were drafted, and the most important deliverable was the establishment of CLAUDE.md guardrails. The config-driven principle, separation of concerns, and leakage prevention rules — the “constitution” that all subsequent AI agents would follow — were established before a single line of code was written. This ordering was intentional: guardrails first, agent execution second.

Phase D: Parallel Implementation (Claude Code — Opus/Sonnet)

In the full implementation phase, each team member served as the “team lead” for their AI agents. Opus and Sonnet were operated in parallel within the Claude Code environment to implement different modules simultaneously. This was the most intensive phase, with three humans each leading AI agent teams.

The PM/Lead Architect’s AI team deployed Opus for architectural decisions (PLE config, adaTT task groups, logit transfer design) and Sonnet for rapid code implementation (generators, adapters, pipeline runner). Critical debugging sessions occurred during this phase: detection of 3 label leakage cases, diagnosis of 4 FP16 NaN root causes, and GPU utilization optimization. Engineer 1’s AI team handled the data ingestion pipeline, HIVE parallel query logic, feature engineering for 10 generators (TDA, HMM, Mamba, Graph, GMM, etc.), and the feature-to-business reverse-mapping registry. Engineer 2’s AI team was responsible for model training, mathematical verification, and knowledge distillation (PLE to LGBM).

The ability to maintain consistency across parallel workstreams was thanks to the CLAUDE.md guardrails established in Phase C and the interface contract verification process. After every parallel work session, it was mandatory to verify that the keys saved by File A matched the keys read by File B, preemptively preventing interface mismatches that could arise during integration.

Phase E: Experimentation + Papers (Claude Code Extension)

During the ablation experimentation phase, Claude Code was used as a real-time monitoring tool. Ablation progress, GPU utilization, and error detection were performed in real time. The PLE toggle bug was discovered through live debugging — the `use_ple=false` setting was altering the expert composition itself, making fair comparison impossible.

Literature research during experiment wait times also proved valuable. While observing that PLE’s `val_loss` failed to converge, a hypothesis was formed through dialogue with Opus that the softmax gate’s competitive nature was hindering convergence among heterogeneous experts. This led to finding the NeurIPS 2024 sigmoid gate paper, providing theoretical grounding. Experiment result analysis and paper writing proceeded simultaneously.

In the paper writing phase, 4 papers (English/Korean), and 22 technical documents were generated and refined through iterative work with Claude. The development story itself was written by reflecting on the project process together with Claude. In this phase, AI served not as a mere text generator but as a thought partner in constructing the meaning of the project.

Documentation Production Scale

AI collaboration extended far beyond code implementation. The on-prem project alone produced 260+ documents (28 design specs, 19 tech references, 16 code reviews, 95 reports, 5 guides), totaling 30+ MB of technical documentation. Many were co-authored with AI or drafted by AI then reviewed by humans. Notably, the “Sonnet Work Verification Report” demonstrates an AI-to-AI review process

where Opus verified Sonnet’s code output, and the “500+ Item Checklist for Claude Code Opus” shows systematic delegation of quality assurance tasks to AI agents.

Memory Bank and Guardrail System

The AI management framework established in the on-prem project was directly ported to the AWS project. A memory-bank system (8 context files: projectbrief, activeContext, progress, techContext, productContext, systemPatterns, tasks, style-guide) maintained cross-session context. `.claude/RULES.md` enforced coding rules, synchronized with `.cursorsrules` so Cursor AI and Claude Code followed identical guardrails. The team even ran automated experiment branches across three AI platforms (`exp/claude-auto-*`, `exp/codex-auto-*`, `exp/vertex-auto-*`), comparing Claude, Codex, and Vertex AI.

Why Claude Code Was Essential

What proved decisive for this project’s complexity was long context retention (1M tokens), cross-session memory banks, and parallel sub-agent execution.

Tracing 3 label leakage cases in sequence was only possible by maintaining context across days of work. After fixing the first leakage (`has_nba` duplicate column), the second (ground truth glob sorting) and third (generator label input) were discovered in the same session because the context of previous fixes remained alive.

Simultaneously diagnosing 4 FP16 NaN root causes (CGC entropy, OT Sinkhorn, Causal DAG, logits) required surveying the entire model architecture while tracing numerical operations in each expert. This was impossible with a file-by-file approach.

Discovering the NeurIPS 2024 sigmoid paper during experiment wait time, connecting it to our observation of PLE softmax non-convergence, and implementing the sigmoid gate — this all happened within one continuous context: experiment analysis to literature search to theory connection to code implementation in a single flow.

Maintaining consistency across 4 papers and 22 technical documents while making simultaneous edits was also a task only possible with an agent that remembered the entire document ecosystem.

Key Patterns Discovered in AI Collaboration

Throughout the project, recurring patterns of AI collaboration emerged. These patterns were not deliberately designed but manifested naturally through actual work.

1. “AI does HOW, humans decide WHAT and WHY”

: AI generates code and text, but architectural decisions are made by humans. The structural isomorphism insight emerged from human-AI dialogue, but the decision to adopt it as a design principle was a human judgment.

2. “Guardrails before agents”

: `CLAUDE.md` was written before code, not after. Just as a constitution precedes legislation, guardrails precede agent execution.

3. “Heterogeneous AI = heterogeneous experts”

: The model’s heterogeneous expert design was directly applied to development tool selection. Each AI tool performed a specialized role, achieving quality and speed unattainable by any single tool.

4. “Memory bank for continuity”

: A persistent file system for session-to-session context preservation was the key mechanism for overcoming AI agents’ greatest weakness: context loss between sessions.

5. “Fail fast with AI”

: Bugs that would take days to find manually — leakage, FP16 NaN, ablation filter failures — were detected and fixed within minutes with AI agents. Fast failure led to fast learning.

6. “AI as thought partner, not code machine”

: The project’s critical intellectual breakthroughs — the structural isomorphism insight, the sigmoid gate hypothesis, identifying the expert collapse problem — all emerged from human-AI dialogue.

Phase	AI Tool	Role
A. Ideation	Gemini	Concept exploration, architecture scanning, cross-disciplinary brainstorming
B. Technical Validation	Claude Opus	Mathematical verification, loss design, leakage analysis, 19 technical docs co-authored
C. Environment Setup	Cursor	GitHub structure, CLAUDE.md guardrails, 6 design documents
D. Parallel Implementation	Claude Code (Opus/Sonnet)	3-person x AI team parallel coding, debugging, 10 generators
E. Experimentation + Papers	Claude Code Extension	Real-time monitoring, literature research, 4 papers + 22 technical docs

Quality Management Strategy

CLAUDE.md Guardrails

To ensure the quality of AI-agent-generated code, a CLAUDE.md file was placed at the project root to establish guardrails. This file serves as system instructions that the AI agent reads at the start of every session.

Core CLAUDE.md Guardrails

- **Config-Driven Principle:** All parameters are read from YAML. No hardcoding.
- **Separation of Concerns:** Adapter handles only data transformation, Runner handles only the pipeline, train.py handles only training.
- **Data Leakage Prevention:** Scaler is fit only on the train split. gap_days mandatory for

temporal splits.

- **4-Stage Code Review:** Compilation check → Interface contract verification → Hardcoding scan → Separation of concerns audit.

Memory Bank

AI agents lose context when a conversation session ends. To address this, a “memory bank” system was introduced. Session progress, design decision rationale, and feedback history were managed as structured markdown files, enabling the AI to quickly restore prior context at the start of each new session.

Interface Contract Verification

The greatest risk when parallel agents simultaneously modify different modules is interface mismatch — where the key names saved by File A differ from the key names read by File B. To prevent this, an “interface contract verification” step was mandatory after all parallel work. Cross-file key/field mapping tables were auto-generated to detect inconsistencies before they reached production.

Technical Challenges and Solutions

Over 20 technical issues arose during development. They are organized here into five categories. Rather than a debug log, each category illustrates a distinct engineering dimension required for large-scale multi-task training on desktop GPUs.

Data Integrity

Data contamination renders model performance meaningless. Label leakage and schema inconsistencies occurred repeatedly in this project, each requiring systematic detection and the construction of defensive guardrails.

Three Label Leakage Cases

Abnormally high performance ($AUC = 1.0$) was observed early in training, uncovering three leakage sources.

- **Duplicate column:** `has_nba_1` existed as a perfect label duplicate (correlation = 1.0). Resolved by `EXCLUDE` before label re-derivation.
- **File loading order:** Glob’s alphabetical sorting loaded `ground_truth.parquet` instead of `benchmark.parquet`. Resolved by moving the file to a subdirectory.
- **Generator input contamination:** GMM and other generators used label columns as input. An auto-exclude mechanism for `label_cols` was added.

A `LeakageValidator` was added as a mandatory pre-training step, and validation rules were documented in the `CLAUDE.md` guardrails to prevent recurrence.

apply_ablation Schema Not Updated

The ablation filter successfully removed features from tensors, but `feature_schema["columns"]` still listed the original 316 columns, so the model received identical 316-dimensional input every time. The fix required `apply_ablation` to simultaneously update `columns`, `num_features`, and `feature_group_ranges` alongside tensor modification.

Numerical Stability

Mixed precision training doubles throughput, but the narrow representable range of FP16/BFloat16 triggers NaN propagation. Four underflow incidents and two conversion errors occurred.

FP16 Underflow and NaN Propagation

During Phase 2 AMP training, FP16 underflow in CGC entropy, OT Sinkhorn, Causal DAG, and logit computations caused NaN to propagate. Small gradient values fell outside the FP16 representable range.

BFloat16 NumPy Conversion and GradScaler

NumPy does not support BFloat16 as a dtype, so `.numpy()` calls on BFloat16 tensors failed and all validation metric calculations broke. A `.float()` cast before every `.numpy()` call was standardized across the pipeline. Additionally, when every batch in Phase 2 produced NaN, no backward pass was called, and `scaler.step()` raised “No inf checks were recorded.” A backward-count guard was added to skip the step when the count was zero.

Infrastructure

On a single desktop GPU, driver conflicts, background processes, and network restrictions can reshape the entire experimental design.

Docker GPU Passthrough and Zombie Containers

Docker GPU passthrough on Windows proved unstable due to CUDA version mismatches, leading to a switch to local Python development. Separately, unterminated zombie containers occupied GPU memory and reduced training speed to one-third of normal.

torch CPU/CUDA Version Conflicts

Installing SageMaker SDK v3 (3.7.0) silently replaced torch with a CPU-only build, disabling GPU training entirely. Pinning to SageMaker v2 (2.257.1) resolved the issue. In conda, CPU/CUDA build conflicts also recurred; the cache was purged and the environment rebuilt with explicit CUDA version pinning.

torch Cache Recovery and Ollama GPU Occupation

The government office firewall blocked `download.pytorch.org` (403 Forbidden). A cached `pytorch-2.5.1-cuda12.1` package in conda’s `pkgs/` directory was manually copied to restore the environment. Separately, Ollama auto-started and consumed 2GB of VRAM. On a 12GB card, a single background process directly constrained batch size choices.

VRAM Spillover Analysis

At batch 6144: 12GB dedicated + 11GB shared GPU memory = 23GB total, 10 hours per scenario. At batch 2048: 9GB + 0.1GB = 9.1GB, 2 hours per scenario. Shared GPU memory traverses PCIe at 10–20x slower than dedicated VRAM. This quantitative analysis determined the optimal batch size.

Lambda 250MB zip Limit and Migration to Container Image

Serving dependencies grew organically during the Paper 2 build. `lightgbm` (LGBM student models), `lancedb` (recommendation cases and diagnostic store), `duckdb` (VSS search), and `pyyaml` were added in sequence, and the layer-based bundle exceeded Lambda’s 250MB zip ceiling.

Three attempts failed before Container Image succeeded. First, a merged-layer deployment threw `libgomp.so.1 missing at runtime` — `lightgbm`’s OpenMP dependency was absent from the Amazon Linux base. Second, the “pre-download wheels locally and inject via `pip install --no-index --find-links`” path failed with SSL verification errors inside the Docker build network. Third, a stale `credsStore: desktop` entry in `~/.docker/config.json` on Windows blocked ECR login; only after removing it did `docker push` succeed. Finally, the default build produced an OCI manifest that Lambda refused; `docker build --provenance=false` forced a Docker V2 manifest and the image finally launched.

The resolution was to migrate to a Lambda Container Image (10GB limit, Python 3.10 base). Lesson: when serving dependencies accumulate quietly, the zip+layer 250MB ceiling must be checked preemptively. On Windows, Container Image migration takes roughly a day of iteration, walking through credential, SSL, and manifest compatibility in order.

LanceDB Design Error — From Raw Feature Dumps to Recommendation Case Accumulation

The initial Paper 2 design stored all 349-dimensional raw feature vectors $\times$ 1M customers in LanceDB. Runtime footprint hit 1.4GB per snapshot, growing linearly with each periodic refresh.

The blunt self-challenge — “why are you storing every customer in Lance?” — forced the diagnosis into focus. LanceDB’s value is in **accumulating outcome-bearing cases over time** — not in mirroring population-wide feature state as a snapshot. The correct unit is “one recommendation case = one inference log entry.”

The redesign had three parts. First, a `recommendation_cases` table is written after each Lambda invocation: `user_id`, `timestamp`, 13 task probabilities, L1 reasons, FDTVS scores. Second, `DiagnosticCaseStore` and `TemporalFactStore` were consolidated onto the same LanceDB instance, introducing no new database dependency. Third, the raw feature matrix dump was removed — it can always be recomputed from the source parquet on demand.

Lesson: the right question when adopting a vector DB is “**what do we accumulate over time?**”, not “**what current state do we mirror?**” The first pattern leaves useful traces for audit and A/B analysis; the second produces a bloated cache of data that already lives elsewhere.

Pipeline Engineering

System-level issues in large-scale data processing and ablation orchestration.

Transition from pandas to DuckDB/cuDF

Processing 941K rows with pandas caused memory spikes. Migrating to DuckDB (CPU columnar) and cuDF (GPU) simultaneously improved memory efficiency and throughput.

NVIDIA Merlin Ecosystem Evaluation and Selective Adoption

We initially attempted to adopt the NVIDIA Merlin ecosystem (NVTabular, HugeCTR, etc.) as a full-stack solution. However, our custom heterogeneous expert architecture — 7 distinct experts (DeepFM, Temporal Ensemble, HGCN, PersLay, LightGCN, Causal, OT) — did not fit well with Merlin’s opinionated pipeline. Merlin is optimized for single-model training workflows and could not easily accommodate an architecture where each expert demands different input formats and computation graphs. In the end, we retained only Merlin’s DataLoader component, adopted cuDF directly for GPU-accelerated preprocessing and feature engineering, and introduced Triton Inference Server for model serving and deployment. This decision reflects a practical engineering philosophy: evaluate full-stack frameworks thoroughly, but only adopt the components that genuinely fit.

Subprocess Pipe Deadlock

The ablation orchestrator used `subprocess.run(capture_output=True)`, but massive stdout exceeded the 64KB pipe buffer, causing a classic deadlock. Redirecting stdout/stderr to files resolved the issue.

Ground Truth File Loaded by Mistake

Glob’s alphabetical ordering loaded `benchmark_ground_truth.parquet` before the actual source data, causing Phase 0 to build features from answer columns. Moving the ground truth file to a subdirectory excluded it from the glob pattern.

Batch Size Mismatch and bash JSON Escaping

`pipeline.yaml` specified `batch_size=2048`, but `run_ablation_manual.sh` overrode it to 6144, triggering VRAM spillover. All settings were unified to a single config source. A separate bash JSON escaping failure was also fixed.

Calibration Pickle Failure — Local Classes Are Not Serialisable by joblib

A distillation job ran on a SageMaker g4dn.xlarge Spot instance. The pipeline flowed through teacher soft-label generation, 7 LGBM student distillation runs, 3 LGBM direct-training runs, calibration, fidelity validation, drift baseline, and model save. The job ran cleanly for 49 minutes through all distillation and calibration fitting, then crashed at the very last step: `joblib.dump(calib, calib_path)`.

The error was: `_pickle.PicklingError: Can't pickle <class`

`'containers.distillation.calibration.calibrate_students.<locals>._LGBMProbWrapper'>`.

The cause was textbook. `_LGBMProbWrapper` — a sklearn-compatible shim around a trained LGBM so that `CalibratedClassifierCV` could consume it — had been defined **inside** the `calibrate_students()` function as a local class. Python’s pickle protocol cannot serialise local classes because they have no importable module path. The wrapper worked fine during fit; the constraint surfaced only when `joblib.dump` attempted to persist it.

The fix was a single move: `_LGBMProbWrapper` was hoisted to module level in `containers/distillation/calibration.py`. The identical job configuration re-ran in 2352 seconds (39 minutes on Spot, no interruptions this time) and completed with `rc=0`.

Lesson: sklearn-compatible wrappers that will be `joblib`-dumped must live at module level, never inside a function. The Python error is cryptic and only emerges at `joblib.dump` time, so hours of training can be thrown away before the problem is visible. A cheap defence is a unit test that pickles every wrapper type used downstream — the same bug is caught in seconds. A second lesson concerns

the economics of Spot: unreliable recovery amplifies the cost of late-stage crashes. Fail-fast checks at job start beat trusting the integration at job end.

Architecture Insights

Ablation experiments and the training process yielded fundamental discoveries about model structure.

PLE Toggle Bug and Ablation Filter Failure

With `use_ple=false`, all 7 heterogeneous experts collapsed into a single MLP, making the baseline comparison unfair. The fix preserved the expert basket and disabled only PLE layering. Additionally, `feature_group_ranges` stored only column-level keys, so the ablation filter's group-level matching never succeeded — all 24 scenarios showed identical AUC (0.913) (from an earlier v3/v4 debugging episode; the final v1 paper reports 23 scenarios). Adding group-level keys resolved it.

GPU Utilization Optimization

Initial GPU utilization was low with batch size 512. Increasing batch size, tuning DataLoader parameters (`num_workers`, `pin_memory`), and pre-saving tensors in Phase 0 improved training throughput. However, the 12GB VRAM constraint capped batch size at 2048 — exceeding this caused spillover into shared GPU memory, degrading rather than improving performance.

Softmax vs Sigmoid Gate Discovery

PLE's `val_loss` froze at 3.702 in Phase 2, while `shared_bottom` (1 MLP) paradoxically outperformed `ple_only` (7 experts). The CGC softmax gate's competitive nature hindered convergence among heterogeneous experts. A NeurIPS 2024 paper confirmed the theoretical superiority of sigmoid gates; implementation is in progress. Gate function selection proved decisive for heterogeneous expert architectures.

Five adaTT Porting Bugs — Discovered by Comparing Against On-Prem Code

Background: In the structure ablation, adaTT consistently degraded performance (sigmoid: -0.006 , softmax: -0.021 , no PLE: -0.004). The problem was not adaTT's design but implementation errors introduced during porting.

Root Cause: A line-by-line comparison against the on-prem (gotothemoon) source code revealed five bugs.

- Gradient extraction frequency:** AWS extracted gradients only at the last batch of each epoch (once per epoch). The on-prem version extracted every 10 steps (17 times per epoch). Fixed by replacing the `_is_epoch_end_step` flag with `global_step % grad_interval`.
- Config load path:** The root-level `adatt` section in `pipeline.yaml` was not being read from `model_config` or `label_schema`. Fixed by adding a `config.get("adatt")` fallback.
- freeze_epoch not passed:** `AdaTTConfig` was constructed without passing `freeze_epoch`, leaving it always `None`. Transfer weights kept adapting unstably until the end of training.
- Loss composition:** The on-prem version applies uncertainty weighting first (normalizing loss scales) and then adaTT transfer. The AWS version used either/or logic, silently disabling uncertainty weighting whenever adaTT was active. With 13 tasks having mismatched loss scales, transfer was dominated by the largest-loss tasks.
- warmup_epochs: 0:** Transfer began immediately while the affinity matrix was still identity (no measurements taken), resulting in meaningless loss sharing from epoch one.

Outcome: sigmoid_adatt AUC improved from 0.5605 to 0.5746 (+0.014). At peak (Ep6) it reached 0.5786, surpassing the sigmoid baseline (0.5771). More decisively, these five bugs account for the entire -0.019 AUC delta that had once been interpreted as “adaTT damaging PLE at 13-task scale” — after the fixes, the adaTT on-vs-off gap collapsed from -0.019 to -0.001 , inside single-seed noise. The earlier reading was an implementation artefact, not an algorithmic finding.

Lesson: Preflight logging (“AdaTT config: warmup=X, freeze=X, source=X”) was added so that config application can be verified before training begins. Had MLflow been in place, a significant portion of this investigation could have been avoided.

Our “adaTT” Was Not Li 2023 — A Naming Drift Discovery

Background: While scoping Paper 3, I re-read the docstring and reference block of `core/model/ple/adatt.py`. We had been using the name inherited from the on-prem codebase without ever checking whether the name and the implementation shared a lineage.

Finding: The docstring cited only two papers: Fifty et al. NeurIPS 2021 (Task Affinity Groupings, TAG) and Chen et al. ICML 2018 (GradNorm). Li et al. KDD 2023 “AdaTT: Adaptive Task-to-Task Fusion Network” was nowhere to be found.

Diagnosis: Our implementation computes $L_i^{\text{adaTT}} = L_i + \lambda \sum_{j \neq i} w(i \rightarrow j) L_j$, where $w(i \rightarrow j)$ is an EMA-smoothed gradient cosine similarity. This is loss-level transfer. Li 2023’s AdaTT, by contrast, is a **representation-level fusion** that stacks a learned gating network over expert activations while preserving a native expert residual. The two algorithms share only the name.

Implication: Paper 1’s “adaTT” narrative was, in effect, an evaluation of a TAG + GradNorm hybrid — not of Li 2023’s mechanism. A single inherited name created a systematic interpretive confusion. A reviewer comparing our results against Li 2023 would have reasonably concluded we had failed to reproduce their method.

Lesson: Code naming must match the citation lineage. When a mechanism is “inspired by” prior work, the docstring must explicitly separate **what was reproduced** from **what was changed**. One name cannot be allowed to paper over five years of divergent research threads.

Paper 1 v1.1 — Algorithmic Finding or Implementation Artefact?

Background: After fixing the five porting bugs and resolving the naming drift, I re-ran `struct_13_ple_sigmoid` and `struct_13_ple_sigmoid_adatt` at 10 epochs with a single seed. We needed to see whether the pre-fix and post-fix numbers told the same story.

Finding: The sigmoid + adaTT AUC moved from 0.6541 (pre-fix, the number Paper 1 originally reported as a -0.019 degradation) to 0.6717 (post-fix). The adaTT on-vs-off gap shrank to -0.001 — well within run-to-run noise. The original claim that “adaTT degrades performance” was no longer supported by the data.

Action: We issued a Paper 1 v1.1 correction commit. The abstract and Section 5.4 were rewritten from “degrades performance” to “has null effect at 13-task scale.” The results table was replaced with the post-fix numbers, and the caption disclosed the five bugs by name. Finding 2 (the “156 task-pair instability” hypothesis) was rewritten to read **“the original attribution was plausible but is not supported once the implementation bugs are fixed.”**

Lesson: What looked like a negative **algorithmic** finding turned out to be an implementation artefact. The responsible response at such a moment is not to retrofit a cleaner narrative after the fact; it is to publish the correction transparently. The original attribution — that TAG-style affinity

becomes unstable at 156 task-pairs — was a reasonable hypothesis, but once the bugs were fixed, no evidence remained to support it. That fact belongs in the record as-is.

Reproducing Li 2023 AdaTT — The AdaTT-sp Experiment

Background: After the Paper 1 v1.1 correction, one question remained. If our loss-level variant was a null effect, would Li 2023’s **original** representation-level adaTT behave differently? Up to that point the two algorithms had shared a name without either being evaluated on the other’s terms. While scoping Paper 3, we decided to implement the original mechanism on our heterogeneous expert basket.

Implementation (AdaTT-sp): Li 2023’s core mechanism is a per-task fusion unit = softmax-weighted sum over experts + a learnable-scalar-weighted **native expert residual**. Rather than stacking a separate layer, we added a `fusion_type: "cgc" | "adatt_sp"` flag to the existing CGC gate: when `adatt_sp` is selected, the mean output of the task’s own task-specific experts is added to the gated weighted sum as a residual, scaled by a learnable `native_residual_weight` (init = 1.0). The `pipeline.yaml` default is `adatt_sp.enabled: false`; it is activated only through the HP flag `use_adatt_sp=true`. Roughly 50 lines of new code, no behavioural change to the main path.

Result (10 epochs, single seed): `struct_13_adatt_sp` achieved AUC 0.6696, Best NDCG@3 0.6825. Against the baseline `struct_13_ple_sigmoid` (CGC gate, AUC 0.6728), the AUC delta was -0.0032 . For reference, the loss-level variant’s delta was -0.0011 , so the **original representation-level mechanism produced a larger drop**, not a smaller one — about three times as large. The Paper 3 scoping hypothesis (“the original might work better”) was refuted on this data.

Lesson: When a heterogeneous expert basket already encodes enough inductive bias, whatever per-task fusion mechanism sits on top — loss-level TAG + GradNorm, or representation-level Li 2023 AdaTT-sp — lands in null-to-negative territory. The more accurate reading is not “which fusion wins,” but **at this scale, fusion augmentation is not needed**. Paper 3’s primary contribution has shifted from “which fusion is best” to **“when does fusion augmentation stop mattering?”**

The M1 Residual Complement Experiment — “Recovering the Unselected” Fails

Background: After the AdaTT-sp failure, one original-form hypothesis from the project’s early scoping remained. “When the PLE gate selects certain experts, might useful signal remain in the ones it down-weighted? What we want is a residual-signal-extraction mechanism that replaces adaTT.” To test this directly — not via cross-task mixing (loss-level adaTT), not via re-injecting the task’s own expert (AdaTT-sp), but as an **intra-task complementary recovery** — we designed M1.

Design: A third `fusion_type = "residual_complement"` was added to `CGCLayer`. The primary gated weighted sum is preserved, and a complement = $(1 - \text{gate_weights})$ is clamped and renormalised to form a complementary weighting, which is then aggregated over `all_outs` as a residual. The final output is gated + $w_r \cdot \text{residual}$ where w_r is a learnable scalar (`residual_recovery_weight`). Off by default in `pipeline.yaml`; activated only through the HP flag `use_residual_recovery=true`. Roughly 30 lines of new code, no effect on existing paths.

Result (10 epochs, single seed): `struct_13_residual_complement` finished at AUC 0.6675 with best AUC 0.6692 at epoch 1. Against CGC baseline (0.6728), $\Delta = -0.0053$ — the **largest drop** among the three fusion variants. More decisively, the peak AUC occurred at epoch 1 and the curve declined monotonically thereafter — as the learnable recovery weight was trained, performance actively worsened. Random initialisation was a less harmful state than the converged weight.

4-way comparison (struct_13 benchmark, 10 epochs, seed=42):

Fusion	Mechanism	Final AUC	Best AUC (ep)	Δ vs CGC
CGC gate	gated weighted sum of selected experts	0.6728	0.6728 (ep10)	—
Loss-level adaTT	cross-task loss mixing	0.6717	0.6733 (ep2)	−0.0011
AdaTT-sp (Li 2023)	per-task own-expert residual	0.6696	0.6714 (ep3)	−0.0032
M1 residual complement	(1 − gate) recovery of unselected	0.6675	0.6692 (ep1)	−0.0053

Per-task analysis: Although the aggregate Δ is at noise level, task-level breakdown revealed three groups. Group A (low gate entropy, 3 tasks: `segment_prediction`, `top_mcc_shift`, `mcc_diversity_trend`) was insensitive to every recovery mechanism ($|\Delta| \leq 0.003$). Group B (high-entropy tasks, 2 cases: `churn_signal` and `will_acquire_lending`) suffered the largest M1 degradation (−0.020 and −0.009). The single positive outlier was `next_mcc` (50-class, base F1 near random at 0.01): $\Delta = +0.005$ across all three recovery variants. The remaining 8 tasks fell within noise ($|\Delta| \leq 0.005$).

Gate-entropy correlation: Task-level gate entropy was extracted from the last CGC layer of the `joint_full` checkpoint and correlated with each variant’s Δ . M1: $r = -0.40$; AdaTT-sp: $r = -0.32$; loss-level adaTT: $r = -0.31$. All three pointed the same direction (higher entropy $\rightarrow$ recovery hurts more), but with $n=13$ and $p \approx 0.18$ the correlation is not statistically significant. Gate entropy therefore cannot be claimed as a structural explanation of recovery benefit. The two outliers — `churn_signal` and `next_mcc` — were better explained by task-specific factors (label construction for `churn`, near-random base rate for `next_mcc`) than by entropy.

Shared failure mode of the three: loss-level adaTT, AdaTT-sp, and M1 all share the same structure — **a gate-derived residual additively injected at the same fusion point as the primary**. When the CGC gate is already near-optimal at AUC 0.6728, inverting the gate or forcibly restoring down-weighted experts reduces to adding noise. The converging conclusion across three experiments is simple: **gate-derived residual has no recovery value**.

Lesson: To improve the architecture, the definition of “residual” has to be decoupled from the gate itself. The candidate directions for Paper 3 are (a) a boosting-style residual path that trains on the primary prediction’s **errors**, (b) a **task-agnostic** global aggregation placed in parallel with the primary rather than added to it, and (c) a self-regulating second-stage gate conditioned on prediction uncertainty. Each of these structurally avoids the failure mode shared by the three rejected variants. Selection of one of these three will drive the next experiment.

ECEB (Error-Conditioned Expert Bank) MV — The Experiment That Invalidated Our Diagnosis

Background: After the M1 failure, the common failure mode of the three recovery experiments (loss-level adaTT, AdaTT-sp, M1 complement) was diagnosed as “a gate-derived residual injected additively at the primary’s fusion point.” To test that diagnosis, we implemented an ECEB minimum-viable design whose residual is derived not from the gate output but from the gate’s **entropy** (uncertainty). By construction, ECEB escapes the shared failure mode of the previous three.

Design (MV-ECEB): A fourth `fusion_type = "eceb"` was added to `CGCLayer`. The primary gated weighted sum is preserved. The recovery path is a **task-agnostic consensus** (mean over all expert outputs), computed independently of the gate. The recovery weight is the product of a per-task

learnable scalar $\sigma(w_t)$ and a per-sample normalised gate entropy $\frac{H(g_t)}{\log} N$. Final output = gated + $\sigma(w_t) \cdot \text{entropy_ratio} \cdot \text{consensus}$. When the gate is confused (high entropy) recovery activates; when the gate is confident (low entropy) recovery approaches zero. Off by default in pipeline.yaml; enabled only through the HP flag `use_eceb=true`. Roughly 50 lines of new code, no effect on existing paths.

Result (10 epochs, single seed): `struct_13_eceb` finished at AUC 0.6665 with best AUC 0.6670 at epoch 4. Against CGC baseline (0.6728), $\Delta = -0.0063$ — the **worst** among the four recovery variants. Unlike M1, the trajectory was not a monotone decline but oscillated in the 0.665–0.667 range; still, baseline was never crossed.

5-way comparison (struct_13 benchmark, 10 epochs, seed=42):

Fusion	Residual definition	Fusion point	Final AUC	Δ vs CGC
CGC gate	—	—	0.6728	—
Loss-level adaTT	cross-task loss mixing	primary loss	0.6717	−0.0011
AdaTT-sp	own task expert mean	primary repr (additive)	0.6696	−0.0032
M1 complement	(1 − gate) complement	primary repr (additive)	0.6675	−0.0053
ECEB (MV)	uncertainty × consensus	primary repr (additive)	0.6665	−0.0063

Revised diagnosis: ECEB decouples the residual definition from the gate output and still fails. The earlier diagnosis — that the common failure mode was “gate-derived residual” — was therefore partially wrong. A more accurate reading is: **the primary output on this benchmark is already near-optimal, and any residual, however it is defined, structurally reduces to noise when injected additively at the same fusion point.** Regardless of how clever the residual definition is (gate inverse $\rightarrow$ uncertainty-gated consensus), $|\Delta|$ grew monotonically with the invasiveness of the intervention.

Narrowing Paper 3 further: All four experiments share the structural pattern “add a residual to the primary representation.” The one remaining candidate that escapes this pattern is A (BRP, Boosting-Residual Path): a residual expert trained on the primary’s **prediction errors**, combined with the primary only at the final prediction stage rather than at the representation. B (TALA) looks parallel in form but ultimately combines at the same point, so ECEB effectively acts as a MV-TALA as well. The next experiment is fixed on BRP.

BRP (Boosting-Residual Path) MV — The First Qualitatively Different Outcome

Background: The fifth experiment. All four prior fusion variants (loss-level adaTT, AdaTT-sp, M1 complement, ECEB MV) failed under one shared structure: a residual injected additively into the primary representation. BRP explicitly escapes that structure — the residual expert produces a **logit residual in output space**, and is trained only on the primary’s detached prediction error (single-stage boosting). The primary representation is never touched.

Design (MV-BRP): A `ResidualExpertBank` module was added, one MLP head per task (hidden=[128, 64], dropout=0.1), input = the last CGC layer’s `shared_concat` (448-dim = 7 shared experts × 64 hidden), output = primary tower’s output dim (1 for binary/regression, N for multiclass). Training discipline: `residual_target = y - "activation"(primary.detach())`, residual loss = `MSE(residual_logit, residual_target)`, weighted at 0.1 in the total loss. The primary tower receives `primary_predictions` separately and trains against ground-truth as in baseline PLE. At inference and evaluation, predictions combine as `primary + sigmoid(lambda_t) dot "residual"` with a per-task learnable scalar λ_t (init = −2.0, i.e. sigmoid $\approx$ 0.12 to suppress the residual early in training). Off by

default in `pipeline.yaml`; activated only through the HP flag `use_brp=true`. About 200 lines of new code, no change to existing paths. Total parameters 3.56M (+0.36M over baseline).

Result (10 epochs, single seed): `struct_13_brp` finished at AUC 0.6650 ($\Delta = -0.0078$ vs CGC, the worst aggregate AUC of the six runs). But `F1_macro` was 0.2117 — the **highest** of all six by a clear margin (+0.0115 over CGC) — and `NDCG@3` was 0.7039 (also the highest, +0.0219 over CGC). `val_loss` finished at 20.87, the lowest of the six. The trade-off was not monotone degradation: the primary metric (AUC) dropped, while ranking and multiclass metrics materially improved.

6-way comparison:

Fusion	Final AUC	F1 macro	NDCG@3	Δ AUC
CGC baseline	0.6728	0.2002	0.6820	—
Loss-level adaTT	0.6717	0.2013	0.6646	−0.0011
AdaTT-sp	0.6696	0.1998	0.6570	−0.0032
M1 complement	0.6675	0.1998	0.6611	−0.0053
ECEB (MV)	0.6665	0.1998	0.6549	−0.0063
BRP (MV)	0.6650	0.2117	0.7039	−0.0078

Per-task trade-off: `next_mcc` (50-class, base F1 near random at 0.01) improved dramatically from 0.0100 to 0.0440 (+340% relative). In the other direction, `churn_signal` — which had the highest binary AUC in the baseline at 0.6868 — dropped to 0.6512 (−0.036), and this single task accounts for most of the aggregate AUC loss. Excluding `churn_signal`, AUC would sit near baseline. The other 5 binary tasks each lost between −0.001 and −0.010.

Mechanism diagnosis: the residual bank takes `shared_concat` as input, and the residual-MSE gradient propagates back into the shared experts (only the residual **target** was detached, not the input). For tasks where primary AUC is already near its ceiling, the shared experts had converged to a primary-supporting optimum; additional MSE pressure pulled them off that optimum. For tasks where primary struggles (`next_mcc`), the residual supplies real signal. In aggregate, BRP functions as a **task-balance regularizer** that redistributes shared-expert capacity from easy tasks toward hard tasks — structurally distinct from the monotone degradation of the prior four experiments.

Assessment: none of the six experiments improved aggregate AUC. But BRP was the first to produce **metric-selective** gains, indicating that a structurally decoupled path — one that does not inject additively at the primary fusion point — can at least reach some metrics that the representation-additive family cannot. Paper 3’s narrative fixes the claim that fusion augmentation on this benchmark cannot improve AUC across the six variants evaluated, and files BRP as a trade-off case (hard-task rescue vs. easy-task capacity loss). More refined variants (`shared_concat detach`, per-task enabling, tuning `residual_loss_weight`) are follow-up scope.

Environment pivot: all six experiments were executed on the local RTX 4070 at roughly 33 minutes per scenario. The next phase (BRP variants, multi-seed, cross-dataset) exceeds the local GPU time budget and moves to SageMaker submission. Local architectural debugging is complete; subsequent work is in the scaling-up phase.

BRP-detached (MV+) — Verifying the Gradient-Entanglement Hypothesis, and Paper 3’s Positive Finding

Background: MV-BRP produced a qualitative trade-off but its aggregate AUC was still the worst of the six variants ($\Delta = -0.0078$). The per-task breakdown showed that `churn_signal` — the highest

binary AUC in the baseline at 0.6868 — fell to 0.6512 under BRP, a -0.036 drop that explained most of the aggregate loss. The proposed mechanism was that residual-MSE gradients, flowing back into shared experts through the `shared_concat` input, were pulling those experts off the primary-supporting optimum they had converged to. Before the SageMaker pivot, one last local variant was run to test this hypothesis with a single-line change: feed `shared_concat.detach()` into the residual bank so that shared experts cannot receive residual-MSE gradients at all.

Design: added `detach_input: bool = False` to `BRPConfig`, default off in `pipeline.yaml`, activated only through the HP flag `brp_detach_input=true`. Forward pass: `brp_input = shared_concat.detach() if self._brp_detach_input else shared_concat`. No parameter-count change, no training-step change, the primary pathway is unaffected.

Result (10 epochs, seed=42, one mid-run restart): `struct_13_brp_detached` reached a final AUC of **0.6721** against CGC’s 0.6728 — $\Delta = -0.0007$, within noise, effectively tied. Best AUC **0.6736** at epoch 8 actually exceeds the baseline by $+0.0008$. F1 macro 0.2075 ($+0.0073$ over CGC), NDCG@3 0.6965 ($+0.0145$, peaking at 0.7315 at ep7), MAE 0.9688 (a small regression loss).

Per-task validation — hypothesis confirmed:

task	type	CGC	BRP (MV)	BRP-detached	verdict
churn_signal	binary	0.6868	0.6512	0.6852	restored ($-0.036 \rightarrow -0.002$)
will_acquire_lending	binary	0.6549	0.6453	0.6553	restored
will_acquire_deposits	binary	0.6534	0.6493	0.6536	restored
will_acquire_investments	binary	0.6754	0.6719	0.6764	restored
next_mcc (50-class)	f1	0.0100	0.0440	0.0356	retained ($+340\% \rightarrow +256\%$)
others (8 tasks)	—	—	within noise	within noise	unchanged

Interpretation: the easy-task AUC loss in MV-BRP was an **implementation artefact** (a `shared_concat` gradient leak), not an algorithmic trade-off. A single `.detach()` line restored churn_signal to 0.6852 ($\Delta = -0.002$, within noise), while retaining $+256\%$ relative rescue on next_mcc. The residual bank’s own parameters carry enough capacity to learn the hard-task correction signal; once shared experts are insulated from residual-MSE gradients, they stay on the primary-supporting optimum and the two paths stop competing.

Final 7-way comparison:

Fusion	Final AUC	F1 macro	NDCG@3	Δ AUC
CGC baseline	0.6728	0.2002	0.6820	—
Loss-level adaTT	0.6717	0.2013	0.6646	-0.0011
AdaTT-sp	0.6696	0.1998	0.6570	-0.0032
M1 complement	0.6675	0.1998	0.6611	-0.0053
ECEB (MV)	0.6665	0.1998	0.6549	-0.0063
BRP (MV)	0.6650	0.2117	0.7039	-0.0078
BRP-detached	0.6721	0.2075	0.6965	-0.0007 (tied)

Paper 3 narrative update: the earlier 6-way reading was “six fusion augmentations on top of heterogeneous-expert PLE + CGC gate all fail to improve aggregate AUC.” With BRP-detached included, the reading becomes more precise. **Representation-additive fusions** (the adaTT family plus ECEB) monotonically degrade AUC. **Output-space boosting with shared-expert gradient isolation** (BRP-detached) instead preserves AUC and improves F1 and NDCG@3. Paper 3 is no longer a pure negative result; it is a mechanism diagnosis plus a structural fix. Six of the seven experiments define what not to touch (the primary representation, additively); the seventh shows how to reach a positive region (output-space boosting + gradient isolation on shared_concat).

Robustness caveat: single seed, single dataset. Multi-seed and cross-dataset validation is reserved for the SageMaker path.

NEAS (Neglected-Expert Auxiliary Supervision) — The First AUC-Positive Finding

Background: BRP-detached tied the CGC baseline on aggregate AUC and improved F1/NDCG, but $\Delta \text{AUC} \approx 0$. For Paper 3 to claim a **positive AUC effect** at all, a different mechanism was needed. The seed came from a brainstorm: “the softmax gate sharpens expert preferences — what if we use the inverse-gate-weighted aggregation of the neglected experts for training?” Among three interpretations, the one chosen was **load-balancing auxiliary supervision** — an explicit regulariser against expert collapse rather than another residual path.

Design (NEAS): the last CGC layer stores its per-task `gate_weights` and `all_outs` live. An inverse-gate aggregation $\text{inv_agg}_t = \sum \frac{1-g_t}{\text{sum}}$ is computed per task and passed through a per-task auxiliary MLP head (128-64-output) that is trained against the primary target y , reusing the primary’s loss functions (`pos_weight` / `class_weight`). The auxiliary loss is scaled by `aux_weight = 0.05` and added to the total loss. Inference ignores the auxiliary head entirely — NEAS is a training-time regulariser only. `NEASConfig` added, HP flag `use_neas`, +0.17M parameters (aux heads only).

Result (10 epochs, seed=42): `struct_13_neas` reached final AUC **0.6739** against CGC’s 0.6728 — $\Delta = +0.0011$, **positive**. The trajectory was **monotone increasing** for all 10 epochs (crossed baseline at epoch 5, rose steadily thereafter). F1 macro 0.2019 (+0.0017), NDCG@3 0.6896 (+0.0076), MAE 0.9605 (essentially tied with CGC’s 0.9601). `val_loss` 25.73 was the lowest among all eight scenarios. Best AUC 0.6739 at epoch 10 — still rising, no overfit.

Per-task pattern — uniform small lift: 11 of 13 tasks improved over CGC. Six of seven binary tasks improved (+0.0004 to +0.0029), `churn_signal` tied, `top_mcc_shift` a tiny loss. Multiclass: `nba_primary` +0.0056 (meaningful), the other two roughly flat. Regression: two of three improved slightly.

Character contrast with BRP-detached:

Trait	NEAS	BRP-detached
Aggregate Δ AUC	+0.0011 (positive)	−0.0007 (tied)
Mechanism	Training-time load-balance regulariser	Output-space error boosting
Effect pattern	Uniform small lift across most tasks	One big hard-task rescue (+256% <code>next_mcc</code>)
Trade-off	Near-zero	Small loss on most tasks
Added parameters	0.17M (aux heads only)	0.36M (residual bank)
Inference overhead	0 (training-only)	non-zero (residual expert forward)

NEAS + BRP-detached — Non-Additive

Background: NEAS and BRP-detached succeed on **different axes** (expert-collapse prevention vs. primary-error correction), so their combination might be additive. Hypothesis: NEAS’s uniform small lift + BRP-detached’s `next_mcc` rescue in a single model.

Design: a new scenario `struct_13_neas_brp_detached` with `use_neas=true` + `use_brp=true` + `brp_detach_input=true`. Both mechanisms activate; no structural change.

Result (10 epochs): final AUC **0.6722** ($\Delta = -0.0006$), best AUC 0.6733 at epoch 7. **NEAS’s +0.0011 gain vanished.** F1 0.2062 and NDCG@3 0.6864 are both below BRP-detached alone. The trajectory peaks at ep7 and oscillates through ep8–10, erasing the monotone-rise pattern of NEAS alone.

Per-task analysis — locating the conflict: what survives the combination is **BRP-detached’s `next_mcc` rescue** (+0.0250 vs +0.0256 standalone). What disappears is **NEAS’s entire uniform binary lift**:

task	CGC	NEAS	BRP-d	Combined
will_acquire_deposits	0.6534	+0.0029	+0.0002	−0.0005 (lost)
will_acquire_lending	0.6549	+0.0021	+0.0004	−0.0000 (lost)
will_acquire_payments	0.6923	+0.0009	−0.0017	−0.0000 (lost)
nba_primary	0.1876	+0.0056	−0.0035	−0.0071 (worse than both)
next_mcc	0.0100	+0.0007	+0.0256	+0.0250 (BRP retained)
top_mcc_shift	0.6312	−0.0002	−0.0011	−0.0031 (additively worse)

Mechanism of the conflict: NEAS’s aux loss pushes shared experts toward **generalists** (every expert must remain useful for every task under inverse-gate reweighting). BRP-detached keeps shared experts on the **primary-supporting optimum** via the primary task loss, while shielding its residual bank from shared-expert gradients with `.detach()`. The two pressures collide on the shared experts: NEAS asks them to generalise, BRP-detached asks them to stay specialised. The shared experts oscillate between the two targets — NEAS’s regularisation benefit is erased, while BRP-detached’s residual-bank error-correction partially survives because its path is isolated from the shared-expert gradient.

Paper 3’s final shape: the 9-way result cleanly defines three regions:

Region	Structure	Δ AUC
Failure (5 variants)	Representation-additive fusion (adaTT family + M1 + ECEB)	−0.001 to −0.008
Positive (BRP-detached)	Output-space boosting + gradient isolation	−0.0007 (tied, large F1/NDCG gain)
Positive (NEAS)	Training-time load-balancing regulariser	+0.0011 (actual positive)
Non-additive	NEAS + BRP-detached combined	−0.0006 (NEAS gain lost)

The two positive regions **solve different problems** — NEAS prevents expert collapse, BRP-detached corrects primary errors — and they are **independent positive findings** that **cannot be stacked**.

Operationally, the choice is per-objective:

- Aggregate AUC lift with uniform per-task improvement → NEAS
- Hard-task (near-random baseline) rescue → BRP-detached

The non-additivity itself is a Paper 3 side lesson: **positive mechanisms that act on different axes of a shared-representation model do not compound, because the shared representation is a finite resource that each mechanism pulls toward its own preferred configuration.**

Natural close of the local experimentation phase: nine scenarios map out the structural boundary of fusion augmentation on this benchmark. Multi-seed robustness, cross-dataset reproduction, and variants aimed at resolving the NEAS × BRP-detached conflict are reserved for the SageMaker path.

Causal Expert $W = 0$ Collapse — The DAG Was Dead Weight

Background: preparing for an Axis-3 study that reinterprets the causal expert’s role, a simple 20-second diagnostic inspected `get_causal_graph()` on existing baseline checkpoints. The expected output was a sparse DAG adjacency matrix learned by NOTEARS regularisation. The actual output was the reason the whole Axis-3 plan paused.

Diagnostic: loading `extraction_layers.0.shared_experts.4.W` from five checkpoints (four local, two upstream on-prem) returned a matrix with Frobenius norm ≈ 0.0001 in every case — smaller than the random-init scale (`randn` × 0.01, expected Frobenius ≈ 0.32). Zero off-diagonal entries exceeded magnitude 0.01. Training was consistently **driving W below its init**, not learning structure:

Checkpoint	W Frobenius	$ W > 0.01$ ratio
struct_13_ple_sigmoid (CGC baseline)	0.0001	0%
struct_13_residual_complement (M1)	0.0001	0%
struct_13_eceb (MV)	0.0003	0%
struct_13_brp_detached	0.0001	0%
upstream on-prem best.pt	0.0001	0%

That the upstream on-prem project also reproduced the collapse ruled out a porting bug: this was a shared structural failure in the original implementation. Paper 1 and Paper 2 claims that “the causal expert learns a DAG” had been inaccurate for over a year.

Root cause — the residual bypasses the DAG: the expert’s forward is

$$z_{\text{hat}} = z + zW^2$$

with z flowing to a downstream `causal_encoder` MLP. The residual z carries the full latent content regardless of W . The task loss therefore has **no structural incentive** to push W away from zero. The NOTEARS acyclicity and sparsity regularisers penalise W away from dense and toward sparse, but neither penalises $W = 0$ — in fact, sparsity is **minimised** there. The global optimum of the combined objective is exactly $W = 0$, and training converges to it reliably.

The gradient also vanishes at the same point. Both the task-loss path via $\partial z_{\text{hat}}/\partial W$ and the NOTEARS reconstruction gradient (added later) have a W factor: $\partial(zW^2)/\partial W \sim 2zW$. Near $W \approx 0$, gradient is near zero, and $W = 0$ is a **saddle point** with no escape velocity at small init scales.

Patch — both parts load-bearing:

1. **NOTEARS reconstruction term.** Zheng et al. (NeurIPS 2018) rely on $\|\mathbf{X} - \mathbf{X}\mathbf{W}\|_F^2$. The compressed-latent analogue goes as a third term in `get_dag_regularization()`:

$$\mathcal{L}_{\text{recon}} = \text{mean}\left((\mathbf{z} - \mathbf{z}\mathbf{W}^2)^2\right), \quad \lambda_{\text{recon}} = 0.5$$

`forward()` caches the current batch’s $\mathbf{z}$ in `_last_z` so the regulariser has a live tensor to consume.

2. **Init rescale.** `randn` $\times 0.01$ sits at a gradient scale of $\approx 10^{-4}$ after the $\partial\mathbf{W}^2/\partial\mathbf{W} = 2\mathbf{W}$ factor — effectively zero. Rescaling to `randn` $\times 0.1$ puts $\mathbf{W}^2$ entries on an $O(10^{-2})$ scale (small enough not to disrupt the residual forward) with non-vanishing gradient.

Either change alone is insufficient: a SageMaker verification run with reconstruction only and init 0.01 returned $\mathbf{W} = 0$ after ten epochs. The second run, with both patches applied, was the first to produce a real DAG on this codebase.

Post-patch verification (SageMaker teacher_full, 10 epochs, softmax gate): $\mathbf{W}$ Frobenius **0.338**, $|\mathbf{W}| > 0.01$ ratio **7.3%** (within the 5–15% range recommended by the NOTEARS paper), $h(\mathbf{W}) = 0$ (exact DAG), diagonal suppressed (no self-loops). Top edges by $\mathbf{W}^2$: `var_23 -> var_13` (0.019), `var_9 -> var_13` (0.009), `var_15 -> var_11` (0.007). `var_13` behaves as a sink hub — the kind of structure a latent DAG is expected to learn.

But task metrics are unchanged:

Run	AUC	F1	NDCG@3	MAE
Pre-patch local softmax ($\mathbf{W} \approx 0$)	0.6729	0.2009	0.6814	0.9598
Post-patch SageMaker softmax ($\mathbf{W}$ learned)	0.6719	0.2042	0.6875	0.9597
Δ	−0.001	+0.003	+0.006	0

Every delta is within the noise band from the 9-way fusion comparison above. The structural bug was real, but its resolution does not, by itself, produce any task metric change.

Interpretation:

- Until now the causal expert was contributing to predictions via `causal_encoder` alone — a perfectly capable non-linear transform whose performance does not require a meaningful $\mathbf{W}$. Adding a meaningful $\mathbf{W}$ perturbs $\mathbf{z}_{\text{hat}}$ by $O(10^{-2})$, which the downstream MLP easily absorbs. **The DAG is decorative in the current architecture**, not functionally routed into prediction.
- Claims that depend on `get_causal_graph()` — attribution, counterfactual probes, reason-code generation — were retrieving essentially random noise before the patch. They are now retrieving a real DAG, but until the prediction path is re-wired to **use** the DAG, the Axis-3 explainability agenda cannot produce a measurable gain. This patch is the precondition for Axis 3, not a standalone improvement.

The honest accounting: a real structural bug was fixed, two full-stack projects received the same patch, and the baseline claim “the causal expert learns a DAG” is now correct. The rest — whether any of this translates into a measurable benefit — is deferred to the Axis-3 studies.

Axis 3 First Experiment — CEH (Causal Explainability Head)

Why CEH first: five Axis-3 candidates are on the table (CEH / CG / CTGR / CRCG / CCP); CEH is the smallest structural footprint of the five — one MLP head plus one MSE supervision signal, no change to task routing (unlike CTGR), no serving-time branching (unlike CG), no cross-module wiring into Paper 2’s reason generator (unlike CRCG). It tests the most basic claim — “now that the DAG

exists post-Finding 8, can a downstream consumer extract something meaningful from it?” — under the cleanest possible conditions.

Design: a small MLP ($64 \rightarrow 64 \rightarrow \text{input_dim}$) on the causal expert’s output produces a per-sample per-feature attribution vector. The training target is the gradient- $\times$ -input saliency baseline of the expert’s own output:

$$\mathcal{L}_{\text{attr}} = \text{MSE}\left(\text{attribution_head}(\text{output}), \nabla_x \text{output} \cdot \sum() \odot x\right)$$

Gradient $\times$ input is computed with one extra forward pass on a cloned, `requires_grad=True` copy of the input, so the main forward graph is undisturbed. Compute overhead is $\approx 14\%$ inside the causal expert (one of seven shared experts, so $\approx 2\%$ model-wide).

The primary prediction path is unchanged. CEH is a training-time regulariser and an inference-time side output: `expert._last_attribution` is accessible at inference for downstream consumers. The audit-log integration (HMAC signing, append-only persistence) is Paper 2 v2 scope.

MV result (SageMaker teacher_full + CEH, 10 epochs, softmax gate):

Metric	Pre-patch ($W \approx 0$)	Post-patch (no CEH)	Post-patch + CEH
Primary AUC	0.6729	0.6719	0.6734
F1 macro	0.2009	0.2042	0.1994
NDCG@3	0.6814	0.6875	0.6842
MAE	0.9598	0.9597	0.9609
W Frobenius	0.0001	0.338	0.366
$ W > 0.01$ sparse edges	0%	7.3%	8.5%
Attribution head trained	n/a	n/a	yes

Attribution head biases (init = 0) moved to $\|b\| \approx 0.08 \pm 0.08$ at layer 0 and 0.03 ± 0.04 at layer 3 — direct evidence the MSE alignment loss was contributing gradient to the head during training.

Observation — CEH slightly reinforces the DAG: W Frobenius grew from 0.338 (no CEH) to 0.366 with CEH, and sparse-edge share from 7.3% to 8.5%. The attribution head’s gradient appears to flow back through the expert’s representation and provide additional structural signal to W itself. Primary AUC moved $+0.0015$ vs the post-patch no-CEH baseline (within noise but in the opposite direction of the earlier -0.001 drift).

What the MV does not verify:

- **Attribution quality.** The head fits its gradient- $\times$ -input target by construction; whether that target is actually a good explanation for regulators / branch staff / customers requires human evaluation on sample cases and comparison against alternatives (Integrated Gradients, DAG-path traversal).
- **Audit-log utility.** The per-sample vector exists at inference but has not yet been routed into an HMAC-signed, append-only audit record that regulators can query. Paper 2 v2 scope.
- **Metric significance.** $+0.0015$ on AUC sits inside the noise band from the 9-way fusion comparison. We do not claim it as a significant improvement.

Paper 3 narrative update: Finding 8 (W -collapse patch) made the DAG **exist**. Finding 9 (CEH MV) demonstrates, at minimum scope, that a **per-prediction output** can be extracted from it — stepping from stage 1 (existence) into stage 2 (routing) of the three-stage progression (existence $\rightarrow$

routing $\rightarrow$ benefit). Stage 3 (benefit) requires audit-log integration, cross-dataset reproduction, and a human-eval pass on the attributions; positive claims are deferred until those land.

CEH Attribution Quality Evaluation — A Global-Importance Collapse

The question that immediately followed MV: “the head trained, but does its output actually carry **per-sample signal**, or does it produce an almost-identical global importance vector for every sample?” A dedicated post-hoc evaluation — 5,000 validation samples, the trained teacher_cek checkpoint, four measurements:

Measurement	Value	Interpretation
Spearman corr. (CEH vs. grad $\times$ input)	mean 0.259, median 0.252	Partial fit to training target
Between-sample / within-sample variance	0.055	Attribution varies much more within a sample than across samples
Top-10 feature overlap (across samples)	0.791	Different samples share $\sim 80\%$ of their top features
Stability under input noise ($\sigma = 0.05$)	Spearman 0.985	Very stable — trivially consistent with a near-global output

Table 1: CEH attribution quality on the Finding 9 MV checkpoint. The low between-over-within ratio combined with high top-10 overlap is the key finding: the head has largely collapsed to a global importance vector with only small per-sample perturbations.

Per-group mass corroborates the flatness. CEH allocates 32.1% of mass to txn_behavior versus 44.7% under grad $\times$ input, and overweights product_holdings (12.6% vs. 5.5%) and product_hierarchy (11.3% vs. 3.8%). The learned distribution is demonstrably flatter than its own training target.

Most likely mechanism: the target itself (grad $\times$ input of the causal encoder’s summed output) has a large sample-invariant component, and a 64-hidden single-layer MLP can capture that component with low loss while ignoring the sample-specific residual. The head is not broken; the target is too flat for a thin MLP to be forced into per-sample discrimination.

What this confirms / rules out:

- Confirms: the infrastructure path validated by Finding 9 MV (functional DAG feeding an attribution consumer) is correct.
- Rules out: the current target design alone cannot produce a regulator-usable per-sample explanation. Downstream Axis-3 candidates that depend on per-sample attribution quality (CRCG in particular) cannot be evaluated against this baseline without target refinement.

Next iteration (v2): introduce `cek.target_mode: "demeaned"`. Supervision becomes grad $\times$ input - batch mean(grad $\times$ input), forcing the head to learn **per-sample deviation** from the global pattern rather than re-learn the pattern itself. Minimal code change (three lines plus a config flag). Directly tests the “target is too flat” hypothesis. Success means the between/within variance ratio increases and top-K overlap decreases; failure routes us to secondary candidates (primary-task gradient target, larger head).

v2 Result — Hypothesis Confirmed, Collapse Resolved

SageMaker teacher_cek_demeaned (10 epochs, everything else identical to the v1 MV run) followed by the same post-hoc evaluation produces:

Measurement	Raw (v1)	Demeaned (v2)	Change
Between-sample / within-sample variance	0.055	0.719	$13 \times$ larger
Top-10 feature overlap across samples	0.791	0.281	65% smaller
Stability under input noise ($\sigma = 0.05$)	0.985	0.953	Still stable
Primary AUC (churn_signal)	0.6866	0.6870	Within noise

Table 2: CEH attribution quality, v1 vs. v2. Two discriminative-power metrics move by more than an order of magnitude while primary AUC is unchanged — the head now learns per-sample deviation rather than reproducing a global importance vector, at no cost to the downstream task.

Hypothesis confirmed: the collapse was a **training-target artefact**, not a head-capacity or architectural limitation. A three-line change plus a config flag restores per-sample discrimination.

Per-group attribution mass also rebalances substantially. `txn_behavior` drops from 32.1% to 18.3%, `gmm_clustering` from 28.5% to 21.4%, while `product_hierarchy` jumps from 11.3% to 30.3% and `product_holdings` from 12.6% to 21.3%. The demeaned head prefers feature groups that carry per-sample distinguishing signal (product taxonomy) over globally high-variance groups that dominated the raw head.

Caveats: Spearman against **raw** $\text{grad} \times \text{input}$ falls from 0.259 to 0.096 between v1 and v2, but this is expected — v2 trains against **demeaned** $\text{grad} \times \text{input}$, so raw is no longer its target. Quality must be assessed with target-independent metrics. Also, per-sample discrimination is **necessary but not sufficient** for regulator-usable explanations; domain validation, human evaluation, and audit-log integration remain outside scope.

Remaining candidate: primary-task-gradient target (replace the causal-encoder sum with a specific task logit gradient, aligning attribution with downstream questions like “why did this customer receive a high churn score?”). The demeaned infrastructure carries over; only the scalar the gradient is taken against changes.

Audit-Log Integration (Paper 2 v2) — CEH Becomes Regulatory Infrastructure

A per-sample attribution vector is only a training metric in isolation. To become regulatory infrastructure it has to answer “**why was this decision made for this customer?**” — GDPR Art. 22’s meaningful-explanation requirement and EU AI Act Art. 13’s transparency obligation. Model-promotion records only track **which model** was in production; individual-prediction audit requires a separate path.

The existing `AuditLogger` (HMAC-SHA256 signatures + SHA256 hash chain + S3 Object Lock WORM storage) already used for promotion decisions gains a `log_attribution` method. One audit entry per prediction:

```
entry = {
    timestamp, operation: "attribution:{model_id}",
    sample_id,          # customer / request UUID
    top_features,       # [{feature, weight}, ...] top-K for human review
    attribution_hash,    # SHA256 of full float32 vector
    prev_hash, hmac     # existing chain linkage
}
```

Three layers of explainability: top-K (human-readable summary), full-vector hash (forensic replay verification), chain linkage (tamper evidence). An auditor re-runs inference on the archived input, recomputes the attribution hash; matching hashes prove authenticity, mismatches prove tampering. The chain verifier also detects selective deletion of inconvenient records.

Code surface is minimal: `CausalExpert.get_last_attribution()` public accessor, `PLEModel.get_ceh_attribution()` to locate the causal expert, and a single `AuditLogger.log_attribution()` method. Five unit tests + an end-to-end smoke test (HMAC emission + chain verification + tamper detection) all pass.

Completeness at this point: CEH infrastructure (Paper 3 Finding 9 v2) + audit-log integration (Paper 2 v2 addition) = a minimal-viable evidence set for per-prediction explainability. What remains is (a) wiring the call into the live serving path (`core/serving/predict.py`), (b) an auditor query UI, (c) domain-expert human evaluation. All of these are infrastructure work rather than Paper 3 follow-up experiments.

Axis 3 Second Experiment — CG (Causal Guardrail): A Negative-to-Positive Pivot

Where CEH answers “why did the model recommend this?”, CG answers the adjacent question “can we trust this recommendation?” — a per-prediction reliability flag that routes suspicious inputs to a fallback or to human review instead of silently passing them through. SR 11-7 known-limitations reporting and EU AI Act Art. 9 risk management both need this. We ran CG as post-hoc analysis on the existing teacher_ceh_demeaned checkpoint, no new SageMaker training.

v1 (W-reconstruction) fails: The first formulation reuses the NOTEARS reconstruction residual from Finding 8. Serving-time score = $\|z - zW^2\|^2$ — low for inputs that fit the learned DAG, high for OOD. Evaluated on 5,000 validation samples plus three synthetic OOD probes (uniform random in $[-3,3]$, column-permuted, extreme-tail percentiles).

Result: all distributions compressed near 1.0 (in-dist median 0.9995, OOD medians 0.9998–0.9972). At p95 threshold TPR was 6.8% / 8.1% / 0% vs. the 5% FPR baseline — chance-level discrimination.

Cause: the learned W is too small ($\|W\|_F \approx 0.36$ on a 32×32 matrix; W^2 is $O(0.13)$ in Frobenius) so zW^2 barely perturbs z . The residual ratio ≈ 1.0 regardless of whether the input fits the DAG. Finding 8’s “decorative DAG” observation propagates directly into CG.

v2 (z-space Mahalanobis) works: If W is too weak, use the causal latent z itself. Compute per-dimension μ, σ from an in-distribution batch; score = $\sum_j \left(\frac{z_{i,j} - \mu_j}{\sigma_j} \right)^2$.

Probe	median	p95	p99
In-distribution	23.6	62.8	268.1
Uniform random	749.5	1200.9	1458.5
Column-permuted	537.4	1076.0	1336.1
Extreme-tail	479.3	479.3	479.3

Table 3: CG v2 z-space Mahalanobis score distributions. Every OOD median lies above the in-dist p99 — near-perfect separation.

At the p95 threshold (62.8), all three OOD types hit **TPR 100%, FPR 5%**. A single formulation switch makes the guardrail functional.

CG works via the causal latent, not the learned DAG weights. This has implications for the remaining Axis-3 candidates: both CTGR (task-group router) and CRCG (reason-code generator) assume the **learned W** carries useful structure. Finding 10 is direct evidence that W is present but not strong enough for structural downstream uses. CTGR/CRCG will inherit the same weakness unless either W is amplified first (larger init, stronger recon lambda, DAG-routed residual path) or the candidates are redesigned to draw from the latent — mirroring the v1→v2 pivot here.

Cost: \$0 (post-hoc analysis, no training).

Finding 11 W-Amplification — “Decorative DAG” Is a Training-Choice Artefact

Finding 10 left two hypotheses open: (a) the learned W is simply too small to drive reconstruction-based discrimination, or (b) the $\|z - zW^2\|$ formulation is structurally limited regardless of W ’s magnitude. Hypothesis (a) is cheap to test — retrain the single `teacher_ceh_w_amp` scenario with `w_init_scale: 0.1 -> 0.3` and `recon_lambda: 0.5 -> 2.0`, everything else identical to the demeaned baseline.

Metric	Baseline	W-amp	Change
$\ W\ _F$	0.363	5.028	$13.9 \times$
Active edges ($ W > 0.01$)	8.5%	59.5%	$7.0 \times$
Max $ W_{ij} $	0.11	0.77	$7.0 \times$
Primary AUC	0.6870	0.6865	within noise

Table 4: W-amplification training result. Every structural measure of the adjacency matrix grows by roughly an order of magnitude at zero primary-task cost. Finding 8’s “decorative DAG” is a training-choice artefact, not an architectural constraint.

CG v1 re-evaluated on the amplified checkpoint (identical OOD probes):

Probe	Baseline TPR	W-amp TPR
Uniform random	6.8%	22.7%
Column-permuted	8.1%	18.6%
Extreme-tail	0.0%	0.0%

Table 5: CG v1 OOD detection (FPR fixed at 5%). The two variable probes improve from chance to weak-but-nonzero; extreme-tail stays at zero regardless of W magnitude.

Extreme-tail’s stubborn 0% reveals the real structural limit. That probe sets every sample to the same vector, so z is identical across samples and so is the residual — a per-sample score has no vantage point on the input **distribution**. No amount of W scaling fixes this; the formulation itself has no distribution awareness. CG v2’s Mahalanobis is explicitly a distance from a reference distribution and therefore handles this case trivially (100% TPR).

CG v2 unchanged at 100% TPR for all three probes — latent-space discrimination doesn’t depend on W and is unaffected by amplification.

Settled:

1. W is not architecturally doomed. Init $0.3 + \lambda_{\text{recon}} = 2.0$ grows it an order of magnitude with zero primary-task cost.
2. Finding 10’s CG v1 failure was **partly** small- W (now contributes 15-20 TPR points) and **partly** a formulation ceiling.
3. Latent-space CG remains dominant; W -based guardrails are a supplement, not a replacement.

Open:

- Does amplified W actually enable CTGR / CRCG / CCP, or do they hit similar formulation ceilings? Finding 11 only establishes the **precondition** (W is usable), not the answer.
- Does more aggressive amplification (init 0.5, $\lambda_{\text{recon}} = 5$, removed sparsity) push v1 to usability or crash primary AUC? Not tested here.

Updated recommendation: run the remaining Axis-3 candidates **on a W -amplified teacher** and in each case evaluate a latent-based alternative alongside. Treat the latent version as the baseline to beat; do not assume W -based variants are preferable.

Cost: \$0.13 (10-epoch spot g4dn, 985s billable).

Design Philosophy: Where Is the Science?

“The Audience of Persuasion Is Always a Person”

The end consumers of a financial recommendation system are not algorithms but people. Customers ask “why this product?”, branch staff must explain recommendation rationale, and regulators verify the model’s decision-making process. A single probability value cannot persuade any of these three groups. Therefore, the criterion for every design decision was: “Can we explain the reason?”

The 2-Axis Decomposition Framework

The core structure of the architecture is based on a 2-axis decomposition: **Financial DNA** (who is the customer?) $\times$ **Data Modality** (what form is the data?). The Financial DNA axis consists of 4 task groups: product holding probability, next product, customer value, and churn risk. The Data Modality axis is composed of heterogeneous experts corresponding to 5 feature types: state, snapshot, time-series, hierarchy, and item. The intersection of these two axes defines the entire model’s learning structure.

The Expert Collapse Problem and the Need for Heterogeneous Experts

When homogeneous MLP experts (e.g., identically structured 3-layer MLPs) are used, **expert collapse** occurs during training — the gating network selects only one expert while the rest go effectively unused. This problem has been confirmed in large-scale experiments at Pinterest and Kuaishou. Structurally different heterogeneous experts (LightGCN, Causal OT, TDA, GMM, etc.) operate on different input spaces with different computational methods, making it impossible for them to converge to identical representations — structurally preventing collapse.

The Dual Role of Features: Prediction Material and Explanation Vocabulary

Features are not merely inputs for prediction performance; they also serve as vocabulary for explaining recommendation rationale. Even a feature with marginal AUC contribution holds irreplaceable value as explanation vocabulary if it enables statements like “this customer’s consumption entropy is high, indicating diverse product experience” at the branch level. For this reason, features were never removed solely based on predictive performance criteria.

From Economics to Data Science

The design philosophy of this project originates from the PM/Lead Architect’s intellectual journey. Trained in economics and decision science, then moving through financial engineering to data science, a fundamental question emerged: **“Where is the science in data-driven methodology?”**

Economics has accumulated centuries of scientific methodology — hypothesis formulation, theoretical frameworks, falsifiability. But economics itself borrowed tools from other disciplines: general equilibrium theory drew from thermodynamic equilibrium in physics, game theory from combinatorics in mathematics, econometrics from statistics. Economics is not uniquely scientific; rather, **cross-disciplinary tool transfer is a universal pattern of scientific progress**. Many Nobel laureates in economics came from physics and mathematics — Samuelson (thermodynamics to economic equilibrium), Black-Scholes (heat equation to option pricing), Nash (fixed-point theorem to game theory), Mandelbrot (fractals to financial volatility). Economics’ most powerful tools came from other sciences.

But moving from financial engineering to data science, scientific rigor gradually faded. Machine learning models were at least mathematically transparent — OLS minimization, SVM margin maximization, decision tree information gain — one could explain *why* they work.

Deep learning changed this. “Neural networks” is a metaphorical name rather than a deep study of actual neural architecture, and the answer to “why does this weight have this value?” is merely “the data trained it that way.” Science requires explanation. An entire field — philosophy of science — emerged to discuss falsifiability, paradigm shifts, and degenerating research programs. Current deep learning approaches felt more like *engineering than science*.

Structural Isomorphism: Bringing Science Back

The answer to this question was *structural isomorphism*.

Humanity has already discovered powerful scientific methodologies across disciplines over centuries. Chemical kinetics, epidemiology, information theory, topology — each represents frontier knowledge about what insights and causal relationships can be extracted from given phenomena.

If we properly recognize the structure of our problem (understanding financial customer behavior), we can import solutions from other disciplines that have already solved structurally equivalent problems. Just as Shannon brought Boltzmann’s thermodynamic entropy to information theory, and Black-Scholes derived option pricing from the heat diffusion equation.

This is the fundamental motivation behind introducing features from 11 academic disciplines and designing heterogeneous experts specialized in each discipline’s mathematical tools. Not “let’s create many features,” but **“what scientific questions should we ask?”** was the starting point of design.

The Place of Scientific Methodology in Recommendation

This project demonstrates that recommendation systems can be grounded in *scientific understanding* rather than mere pattern matching. Not every recommendation needs to end at “similar people bought this” (correlation); it can reach “this customer’s consumption dynamics indicate this product is appropriate” (causal explanation).

Pearl’s causal inference, Friedman’s Permanent Income Hypothesis, Boltzmann’s statistical mechanics — these scientists answered “why?” in their respective fields. Our heterogeneous expert architecture

brings their tools to financial recommendation through structural isomorphism, attempting to endow recommendation systems with scientific explainability.

Key Lessons

“AI Writes the Code, but Design Decisions Remain with Humans”

AI agents generate code at remarkable speed, but architecture decisions, data leakage judgments, and cost optimization strategies depend on human domain knowledge and experience. AI excels at “how,” but “why” and “what” must be defined by humans.

*“The most dangerous moment is right after AI produces ‘plausible-looking code.’
If you stop critically reviewing at that point, technical debt accumulates.”*

“AI Coding Without Guardrails Creates Technical Debt”

When AI is given free rein to code without a CLAUDE.md, hardcoded values proliferate, concerns become entangled, and untestable structures emerge. Guardrails do not restrict AI’s productivity — they channel it in the right direction.

“The Mixture-of-Experts Philosophy Applies to Development Methodology Too”

The core philosophy of the PLE architecture — Mixture of Experts — was applied to the development methodology itself. Gemini specialized in broad exploration, Opus in deep analysis, Cursor in rapid environment setup, and Claude Code in implementation. Assigning roles aligned to each tool’s strengths proved more effective than having a single AI tool do everything.

Results

System Built

Recommendation System

- 13-task multi-task learning
- 7-expert PLE network
- adaTT adaptive inter-task transfer
- Uncertainty weighting (Kendall et al.)
- Three logit transfer modes

Infrastructure and Experimentation

- 23 ablation scenarios (14 joint feature+expert + 9 structure cross)
- AWS SageMaker spot instances
- Phase 0 (CPU) + Phase 1~2 (GPU) separation
- Config-driven pipeline architecture

Documentation

A total of nine technical documents were produced throughout the project: architecture overview, pipeline guide, expert details, feature reference, PLE+adaTT reference, Causal OT reference, distillation reference, temporal reference, and regulatory framework — all authored in Typst.

Papers

Two papers have been prepared: one covering the heterogeneous expert PLE architecture and ablation study (Paper 1), and another covering the recommendation reason generation pipeline, ops/audit agents, and regulatory compliance (Paper 2). Both are available in the project repository in English and Korean.

Expert Specialization Revealed by Ablation

Analysis across 23 ablation scenarios (14 joint feature+expert + 9 structure cross) clearly demonstrated task-type-specific expert specialization. LightGCN showed the greatest contribution for multiclass tasks (next product prediction), while the Causal expert excelled at regression tasks (customer value estimation). This empirically validates the heterogeneous expert design.

On-Premises Operational Results

The on-prem system self-assessed coverage of 85% of the Financial-Sector AI Risk Management Framework (AI RMF) 24 items (11 full + 9 partial), with remaining gaps being organizational decisions rather than technical implementations. 12 regulatory compliance modules were implemented: AI notification, right to refuse, human reprocessing, fairness monitoring, conflict-of-interest prevention, herding detection, prompt injection defense, safety documentation, model cards, audit trail, consent management, and quality monitoring.

Evaluation Metric Framework

Gold standard metrics were established by task type: AUC for binary classification, Macro F1 for multiclass classification, and MAE for regression. This prevents the error of comparing all tasks with a single metric and ensures rigorous evaluation aligned with each task’s characteristics.

Ops/Audit Agents — “AI Analyzes, Humans Decide”

The most ambitious design challenge in this project was building an autonomous pipeline diagnostic agent system, separate from recommendation reason generation. Core question: “Can a small team maintain regulation-compliant AI operations without dedicated MLOps staff?”

Design Process: From Conversation to Architecture

The design evolved progressively through continuous dialog in a single session:

1. **“Let’s add agents”** → Ops/Audit separation → async decoupling rationale (latency, regulatory independence, fault isolation)
2. **“What model level?”** → “Rule engine handles 95%” → deterministic engine + LLM dialog interface separation
3. **“On-prem?”** → “Core features work without Bedrock” → on-prem baseline + AWS Bedrock extension
4. **“Hallucination risk”** → 3-agent independent voting → **“Delphi has convergence bias”** → 2-Round hybrid (independent voting + sequential deliberation) + minority report preservation
5. **“Cases accumulate into knowledge”** → Diagnostic Case Store (LanceDB) → similar search + statistics + resolution tracking
6. **“Korean models?”** → Claude Sonnet (AWS, L2a rewriting) + Exaone 3.5 (on-prem) → per-task optimal model assignment

Repeatedly asking “What’s the weakness in this design?” at each step resulted in a 15-section, 3,800-line design document and 21 files of ~4,800 lines of implementation completed in a single session.

Key Design Decisions

Deterministic Engine with LLM on Top

95% of the agent runs on a Python rule engine (if-else, thresholds, pattern matching). 48 checklist items are auto-evaluated, generating reports in finding + likely_cause + suggested_action format. LLMs are used only for discussing “how to interpret these numbers” with operators.

The core reason for this separation: LLM non-determinism is a risk in audit contexts — same input may produce different diagnoses. The rule engine establishes facts; the LLM discusses interpretation of those facts.

Minority Report: “Missing a Signal is Worse than a False Alarm”

Minority opinions (1/3) in the 3-agent consensus are structurally preserved. Initially considered pure Delphi (sequential deliberation), but identified the problem of “later agents conforming to earlier opinions.” Final design: Round 1 locks minorities via independent voting, Round 2 only strengthens arguments — minorities are never deleted.

The name was inspired by the film *Minority Report*: one of three precogs makes a different prediction.

Consensus Semantics: “PASS Requires Unanimity by Default”

When we built the 3-agent consensus pipeline as three parallel Bedrock Claude Sonnet 4.6 calls, the initial `ConsensusArbiter` classified verdicts by simple majority: 2/3 PASS meant PASS, 1/3 meant FAIL. It looked like common sense.

One line broke that premise: “2 PASS + 1 WARN — that one also has to surface as a minority report.” If majority voting silently absorbs the WARN, there is no way to answer “why was that warning buried?” at regulatory reporting time. Dissent must not be absorbed — it must **surface**.

The redesign is three lines:

1. PASS requires a 3/3 unanimous vote — zero dissent tolerated
2. Any WARN or FAIL from any of the three agents escalates the overall verdict to WARN
3. The `minority_report` field **always** preserves the reasoning of every dissenting agent — never deletable

Lesson: In regulated financial ML, consensus semantics is a **compliance-sensitive design decision, not a statistics question**. The pattern SR 11-7 model risk management expects is “unanimous for PASS, any dissent escalates.” Majority voting feels natural in research, but from an audit perspective it forces you to justify, every time, why a minority opinion was discarded. The default should lean toward the precautionary threshold — the consensus-layer version of our project-wide principle that missing a signal is worse than raising a false alarm.

Built ≠ Wired: “Implemented, but Nothing Calls It”

During Paper 2 / Lambda end-to-end verification, the same pattern surfaced again and again — **the class was complete, but nothing at runtime was calling it**.

- **Champion-Challenger gate**: `ModelCompetition.evaluate()` in `core/evaluation/model_competition.py` was fully implemented — invoked only from tests. `submit_pipeline.py` bypassed it and auto-promoted on fidelity pass.
- **ConsensusArbiter**: implemented, but the initial `OpsAgent` / `AuditAgent` never invoked it.
- **DiagnosticCaseStore**, **TemporalFactStore**: classes defined in both paper design and code; Lambda never called them.
- **FDTVSScorer**, **SelfChecker**: built, but never wired into Lambda `predict.py`.
- **recommendation_cases LanceDB table**: schema defined; Lambda never appended a row.

The trigger was two short questions: “Did you do Paper 2 §5.10?” and “Do the Ops/Audit agents actually reference LanceDB?” Answering them meant running an audit, and **seven unlinked features** surfaced. The fix was a single commit: `feat: Connect all 7 unlinked features to Lambda + agents`.

Lesson: **Implementation is not integration**. AI-augmented development produces modules quickly but routinely skips the wiring phase. Every new module needs a checklist entry — “who calls this, and is the path actually exercised by end-to-end tests?” And a periodic **module inventory vs. runtime**

call-graph audit is non-negotiable. The faster the generation rhythm, the shorter the wiring-audit cycle has to be. “Built” is a weaker claim than it feels; only “wired and exercised” counts.

Feature Reverse-Mapping Pipeline Completion

Four critical gaps in recommendation reason generation were fixed:

1. English fallback replaced with Korean default templates
2. 12+ missing feature prefix-to-group mappings added
3. ReverseMapper integrated as Level RM fallback in InterpretationRegistry
4. `generate_ll()` connected to InterpretationRegistry via 3-tuple enrichment

These fixes ensure L1 reasons include Korean interpretations with IG direction and task context.

Implementation Method: Parallel Sub-Agent Execution

Design document → implementation plan → Phase-by-phase Sonnet sub-agent parallel execution → main agent (Opus) review. Pre-req (4) → Phase 0 (3) → Phase 1+2 (5) → Phase 3+4 (4) → Phase 5 (2) → gap fixes (5), 6 rounds of parallel execution total.

Each round: `py_compile` + `yaml.safe_load` + interface contract verification + hardcoding scan — maintaining a rhythm of “build fast, verify thoroughly.”

Deliverables

- **Design document:** 3,861 lines (Typst) + 1,168 lines (Markdown) + on-prem handoff 430 lines
- **Implementation:** 21 Python files, ~4,800 lines
- **Configuration:** `agent.yaml` + `agent_tools.yaml` (38 tools) + `checklist.yaml` (53 items)
- **Document updates:** Paper 2 both versions + typst 10 files + design 6 + guides 3 = 20+ files
- **Diagrams:** 3 Paper 2 placeholders → fletcher diagrams, 15 docs/typst ASCII → fletcher conversion
- **Translation:** 5 docs/typst/en tech_ref files Korean → English full translation

PaperClip Adoption (2026-04)

PaperClip (2026.3, GitHub 30K stars) has a “zero-human company” philosophy that conflicts with our principles, but three operational mechanisms were worth adopting:

1. **Heartbeat Pattern:** Agents wake periodically to run checklists, but return `HEARTBEAT_OK` and sleep again if nothing changed. Applied to our CP5 (5min), CP6 (1h) periodic checks.
2. **Budget Cap (Prepaid Debit Card Model):** Per-agent monthly token limits, 80% soft warning, 100% hard stop. The key insight is **graceful degradation** — when budget is exhausted, only LLM calls are blocked while the rule engine continues. This naturally aligns with our “on-prem baseline operates without LLM” design.
3. **Full Tool Trace:** Every `ToolRegistry.call()` is automatically logged, enabling complete reproduction of “what tool calls produced this diagnosis.”

LangMem’s **prompt self-improvement** was deliberately not adopted — from an audit perspective, “who approved this prompt?” cannot be answered.

Memory Framework Adoption (2026-04, follow-up)

Immediately after PaperClip implementation, we reviewed other memory frameworks (Mem0, Zep/Graphiti, Letta, SuperLocalMemory). Again, we **did not adopt any framework wholesale** — only four patterns were selectively incorporated.

Zep/Graphiti Temporal Knowledge Graph

“What was customer A’s state at 2026-03-15?” — answering this audit query requires the model version, features, and verdicts from that moment. Rather than joining distributed components, we built `TemporalFactStore` with a `(entity, attribute, value, valid_from, valid_to)` schema that resolves this as a single filter. Shares the same LanceDB instance as `DiagnosticCaseStore`.

SuperLocalMemory Mathematical Decay

“Is a 3-year-old drift resolution approach still valid?” — no. Recent cases are more relevant. But **deleting** old cases violates the 7-year retention requirement. Solution: preserve originals but adjust **search weights only** via $\exp(-\frac{age}{\tau})$ with a 90-day half-life default. Solved with 30 LOC added to `DiagnosticCaseStore.search_similar()`.

Mem0 Fact Compression

`InterpretationRegistry` provides feature-level Korean interpretations, but lacked **customer narrative profiles**. Without facts like “this customer prefers deposits and is risk-averse” in the L2a prompt, Claude Sonnet generates reasons without context. We implemented `FactExtractor` rule-based — 15 rules defined in YAML config, safely evaluated via Python `eval()` with a sandboxed `__builtins__`. **Zero LLM calls.**

Letta Recall Memory

Solved the problem of `BedrockDialogSession` losing dialog history on session termination via DynamoDB-backed `DialogRecallMemory`. Past conversations are retrieved by embedding search (or keyword fallback) and injected into the system prompt.

A Critical Wiring Bug Found in Quality Audit

After implementation, an interface contract audit found **one critical bug**: `generate_ll1()` retrieved `customer_facts` but only attached them to a local variable; they never reached the L2a path through SQS. The entire M-3 chain was a **silent no-op**. Fixed by re-querying facts in `get_best_reason()` and injecting them directly into the SQS context dict.

The lesson: **even when sub-agents implement individual files correctly, connecting data flows between files is a separate task**. Main agent’s final verification is essential.

Data Integrity Audit: v3 to v4

(2026-04-10/11)

“An AUC of 0.98 on an imbalanced multi-task tabular setup is not a breakthrough — it is a red flag. The model is not learning; it is inverting a bucket function it was handed as input.”

Deterministic Leakage Discovery: 18 → 13 Tasks

Four tasks were removed after a leakage audit traced what the model was actually learning.

`income_tier` was a direct bucket of `income`; `tenure_stage` was a bucket of `tenure_months`; `spend_level` was a bucket of `synth_monthly_spend`; `engagement_score` was the linear combination $0.3 \cdot \text{is_active} + 0.4 \cdot \text{freq} + 0.3 \cdot \text{num_products}$. All four were model-input features or trivial transformations thereof.

The audit was triggered by implausibly high validation numbers: `income_tier` AUC = 0.98, `tenure_stage` F1 = 0.98. On a heavily imbalanced, multi-task tabular setup, numbers at this level are not a signal of skill — they are a clear indicator of leakage or trivial reconstruction. A leakage audit was immediately initiated. It traced the cause to the label definitions themselves: each of the four tasks was a deterministic bucket or linear transform of features already present in the model’s input. The CLAUDE.md principle “labels derived by deterministic transformation of input features must not be used as tasks” was already written; the audit confirmed it had not been applied to the task list. Five were removed (including `has_nba`, which was folded into `nba_primary`). The task count fell from 18 to 13, and the remaining tasks represent genuinely uncertain predictions: product acquisition, churn, next-MCC, spend-shift, and similar outcomes that require the model to learn something non-trivial about customer behavior.

The broader lesson: leakage reviews should target **label definitions**, not just feature pipelines. A perfectly clean feature pipeline can still be undermined by a label that is a function of those same features.

Synthetic Data Iterations: v2 → v3 → v4

Getting the synthetic benchmark to produce meaningful label distributions took three iterations and exposed assumptions we had not examined.

v2 used uniform-random MCC assignment and fixed transaction amounts. The result was near-random labels for any MCC-dependent task — the model was predicting outcomes that the data generator had made structureless by construction. v3 introduced persona-weighted MCC (4–5× preference multiplier) and transaction stickiness of 30%. Labels improved, but acquisition tasks remained near-uniform because the persona-to-product mapping was too weak to differentiate customers.

v4 made three sharper changes: MCC preference multipliers were raised to 8–12× (personas now have strongly distinct spending profiles), stickiness was raised to 60% (customers reliably stay in category), acquisition rates were increased across all products, and the mode-shift window was widened to allow more realistic behavior changes over time. The result was meaningful distributions across all 13 tasks — enough variance to reward a model that learns, without label collapse that rewards a trivial predictor.

The pattern across all three iterations: each fix revealed the **next** assumption that was too weak. v2 assumed MCC randomness was fine; fixing that exposed the persona mapping problem. Fixing the persona mapping exposed the acquisition rate problem. Synthetic data design is iterative by nature, and the right signal that an iteration is finished is when label distributions resemble the target domain, not when the generator code looks clean.

HGCN vs. LightGCN Role Confusion

The two graph experts were initially routed to the same type of data: product co-holding relationships. HGCN received `product_hierarchy` (a bipartite product–customer graph), making it functionally identical to LightGCN. Both experts learned collaborative-filtering-style affinity. The on-prem design intention — HGCN learns **tree structure in hyperbolic space**, LightGCN learns **bipartite affinity** — was not reflected in the routing.

The fix required rewriting the `merchant_hierarchy` generator to produce actual MCC L1→L2 Poincaré embeddings (27-dimensional), representing the hierarchical relationship between merchant category groups and their subcategories. `feature_groups.yaml` was updated so that HGCN’s `target_experts` points to the merchant hierarchy group while LightGCN retains product co-holding. After this change the two experts serve genuinely different functions: HGCN encodes taxonomic distance in hyperbolic space; LightGCN encodes purchase co-occurrence patterns via collaborative filtering.

This was a design-intent preservation issue, not a code bug. The generator produced valid embeddings; the routing fed them to the wrong expert. Config-driven routing (`feature_groups.yaml`) is the only sustainable way to manage this — hardcoded routing in adapter or model code would have made the misassignment invisible for much longer.

Infrastructure and Tooling Fixes

Several infrastructure issues were discovered and resolved during this session.

Expert routing granularity. Routing must be specified at feature-group level, not column level. Column-level routing does not survive Phase 0 normalization reordering: when log-suffix columns are appended, column indices shift and the routing silently breaks. Feature-group-level routing uses named groups from `feature_groups.yaml` and is stable across normalization transformations.

Feature group range indexing. `feature_group_ranges` must be built as contiguous blocks using min/max position across all columns in a group. Previous logic broke when `_log` suffix columns were appended after the base columns: the range endpoints no longer bracketed the right indices.

Metric aggregation by task type. Averaging AUC across all tasks regardless of type is misleading. The correct aggregation uses average AUC for binary classification tasks, average F1-macro for multiclass, and average MAE for regression. Mixed-type averaging was producing metrics that obscured model behavior on individual task types.

Ablation result preservation. Results must be archived, never deleted. The `rm -rf` pattern in cleanup scripts was replaced with archive rotation: each run’s results move to a timestamped subdirectory before new results are written. This prevents accidental loss of completed ablation runs.

Windows sleep and subprocess reliability. Windows `sleep` mode interrupts overnight runs. Sleep-on-idle was disabled for long sessions. Spurious subprocess failures (non-zero exit on clean runs) were addressed by adding retry logic with a one-second backoff.

Temperature scaling for LGBM students. Knowledge distillation papers typically recommend $T = 3-5$ for neural student models. For tree-based LGBM students, $T = 1$ was found to be appropriate: LGBM cannot represent the smooth probability distributions that high-temperature softening produces, so the standard neural-student temperature setting was actively harmful. $T = 1$ preserves the teacher’s rank ordering without imposing an impossible distribution shape on the student.

The Bug That Outweighed Every Architecture Decision (2026-04-13)

“The uncertainty weighting fix yielded +0.018 NDCG@3 and +0.031 F1-macro. That is larger than any architectural change we had tried. The model had never been running as designed.”

Uncertainty Weighting: A Silent No-Op

The investigation began with a puzzling trend in ablation results: as architectural complexity increased — from shared bottom to PLE, from no transfer to adaTT — NDCG@3 declined rather than improved. The question that forced a root-cause search was simple: why would a more sophisticated architecture produce worse recommendations?

The answer was in the loss computation. The on-prem uncertainty weighting implementation applies per-task `loss_weight` inside the uncertainty term:

On-prem (correct): `loss = loss_weight * (precision * task_loss + log_var)`

AWS port (bug): `loss = precision * task_loss + log_var / 2`

The AWS port had dropped `loss_weight` entirely and divided `log_var` by 2 as an ad-hoc scaling choice that did not appear in the original formulation. The result: `task_loss_weights` declared in `pipeline.yaml` had been silently ignored since the beginning of experimentation. Every training run had treated all 13 tasks as equally weighted, regardless of configuration. Tasks with large loss magnitudes dominated gradient updates; tasks representing the system’s primary business objectives — next-product prediction and churn — were swamped.

The fix was a single line: restore the `loss_weight` multiplication and remove the erroneous `/2` on the log-variance term. Outcome: +0.018 NDCG@3, +0.031 F1-macro. These gains were larger than any architectural modification tested across the entire ablation study.

The lesson is uncomfortable but important. Infrastructure-level bugs — bugs in the training harness rather than the model architecture — are systematically harder to detect than model bugs. A model that misroutes a feature group will show inconsistent expert activations. A training harness that silently ignores loss weights will train, converge, and produce plausible metrics — all while never optimizing what the configuration specifies.

The Softmax-Sigmoid Reversal

Earlier experiments, run before the uncertainty fix, consistently showed sigmoid gating outperforming softmax. This result was coherent with the NeurIPS 2024 sigmoid gate paper, which argued that competitive softmax normalization hinders convergence among heterogeneous experts. The finding was documented as a conclusion and used to inform subsequent experiment design.

After the uncertainty fix, the result reversed: softmax now outperforms sigmoid on NDCG metrics.

The root cause, in retrospect, is traceable to the broken loss weighting. With all 13 tasks weighted equally, binary classification tasks — which are numerous in the task set — produced gradients that consistently overwhelmed multiclass and regression gradients. Under these conditions, sigmoid gating’s non-competitive behavior was genuinely beneficial: it allowed all experts to remain active and resist being captured by binary task gradients. Softmax’s competitive selection amplified the problem by concentrating expert capacity on the gradient-dominant binary tasks.

With correct loss weighting, multiclass gradients recover their intended strength. Softmax’s competitive routing then becomes protective: by forcing expert specialization, it creates a structural barrier between binary and multiclass gradient flows. The weaker experts for each task type are not pulled off course by the dominant tasks.

Why Sigmoid Wins in Homogeneous MTL — and Softmax in Heterogeneous

The homogeneous MTL literature (2–4 tasks, same task type) favors sigmoid gates because competitive routing among structurally similar experts produces collapse. But the 13-task, 3-type setting in this project is not homogeneous MTL. Binary, multiclass, and regression gradients have incompatible scales and update frequencies. In this regime, softmax’s competitive expert selection acts as a routing firewall — protecting task-type-specific experts from gradient corruption by other task types. The sigmoid literature result does not transfer. The boundary condition matters.

This episode illustrates a broader methodological risk: when a training bug corrupts the optimization process, observed results can point toward incorrect architectural conclusions. The sigmoid preference was a valid adaptation to a broken training environment. Once the environment was fixed, the underlying architectural preference reasserted itself. Without the root-cause investigation, the “sigmoid is better” conclusion would have been carried forward indefinitely.

adaTT at Scale: 13 Tasks, 156 Pairs — And a Retracted Reading

An early ablation showed adaTT reducing AUC by -0.019 against the PLE-only baseline at the 13-task configuration, and for a while this was the evidence behind a narrative of “adaTT structurally failing at 13-task scale.” Two explanations were offered in parallel: a combinatorial one (affinity estimation is noisy across 156 directed pairs with limited gradient history), and a structural one (PLE’s representation-level separation is undone by adaTT’s loss-level re-mixing). That reading is what earlier revisions of this development note carried.

The next session’s line-by-line diff against the on-prem source surfaced five porting drifts — gradient extraction frequency (once per epoch vs. every 10 steps), config loader path (the root-level `adatt`:

block was never read), `freeze_epoch` not wired into `AdaTTConfig`, uncertainty weighting and `adaTT` implemented as either/or rather than serial, and `warmup_epochs=0` as the silent default. After all five were fixed, Sigmoid + `adaTT` AUC moved from 0.6541 to 0.6717, and the `adaTT` on-vs-off delta shrank from -0.019 to -0.001 — inside single-seed noise.

The -0.019 was an implementation artefact, not an algorithmic finding. The earlier reading of a structural PLE + `adaTT` conflict at 13-task scale did not survive empirical retest; the honest current statement is that `adaTT` is a null effect at this scale — no reliable gain, no reliable harm. The combinatorial-affinity and representation-vs-loss-level hypotheses remain theoretically plausible but are not confirmed on this dataset at this size. The reason this episode is kept in the record is plain: architectural conclusions drawn before the bugs are fixed harden implementation debris into permanent “findings.”

GradSurgery: Tested Gradient-Level Alternative (Not Adopted)

The diagnosis of the PLE + `adaTT` conflict motivated a new experiment. Rather than attempting to mix task losses after the fact, the question became: can we prevent gradient corruption at the point where it occurs — during the backward pass? GradSurgery was implemented and evaluated, but ultimately showed no meaningful advantage over the PLE-only baseline; it was not adopted for production.

GradSurgery (Yu et al., 2020) operates by projecting conflicting task gradients onto the normal plane of the interfering gradient, eliminating the component that would reduce another task’s performance while preserving the component that benefits it. The operation is gradient-level, not loss-level.

The architectural fit with PLE is direct. PLE separates tasks at the representation level. GradSurgery protects that separation at the gradient level. Instead of re-mixing across all 156 task pairs, gradient projection is applied at task-type group boundaries: binary tasks as one group, multiclass tasks as another, regression as a third. This is a dual-axis design — semantic grouping (Financial DNA task groups) for PLE routing, technical grouping (task type) for gradient protection.

Two-Axis Expert Architecture vs. Two-Axis Training Design

The model architecture uses a 2-axis decomposition: Financial DNA (task semantics) $\times$ Data Modality (feature type). The training design mirrors this: Financial DNA groups define PLE routing; task-type groups (binary / multiclass / regression) define GradSurgery projection boundaries. The same structural insight that produced heterogeneous experts now produces heterogeneous gradient management.

The implementation challenge is computational. GradSurgery requires retaining the computation graph across all task backward passes to compute pairwise gradient dot products, then projecting and reapplying gradients. This `retain_graph=True` overhead is non-trivial on a 12GB GPU.

VRAM on a 12GB Card: Lessons From GradSurgery

The GradSurgery implementation surfaced a recurring theme from this project: every architectural improvement has a VRAM cost, and on a 12GB desktop GPU, that cost is always visible.

`retain_graph=True` keeps the full computation graph in memory across all task backward passes. For 13 tasks, this is approximately equivalent to holding 13 separate gradient tapes simultaneously. At

batch size 2048 — already the established optimum for the base model — this produced memory pressure requiring reduction to batch size 1024.

Two mitigations proved effective. First, gradient projection at `grad_interval=10` steps (matching the adaTT gradient extraction frequency) rather than every step reduces the number of `retain_graph` calls by 10×. The projection signal remains stable because affinity relationships between task types change slowly. Second, projection across task-type groups rather than all 156 pairs reduces the number of pairwise dot products from $O(N^2)$ to $O\left(G^2 \times \left(\frac{N}{G}\right)^2\right)$ where G is the number of groups — a meaningful reduction for small G .

A separate VRAM issue emerged from the development environment itself. Ollama auto-starts on system boot and loads its language model into GPU memory. On a 12GB card, this consumes approximately 2GB — a significant fraction when batch size decisions hinge on the difference between 9GB and 11GB of active memory. The solution was to disable Ollama auto-start and kill its process before initiating any training run. This became a standard step in the pre-training checklist alongside `LeakageValidator` and preflight logging.

The pattern across all VRAM incidents in this project is consistent: background processes and framework overhead are not visible until a training run begins and memory pressure reveals them. The only reliable defense is a pre-training VRAM audit: `nvidia-smi` before every run, with a hard stop if available memory is below the established baseline.

GradSurgery Outcome: Not Adopted

Despite the theoretical fit and careful VRAM mitigations, GradSurgery did not produce a meaningful improvement over the PLE-only baseline in ablation evaluation. The retained computation graph overhead reduced the effective batch size from 2048 to 1024 without a compensating gain in task metrics. The production configuration therefore disables both adaTT and GradSurgery, relying on PLE’s representation-level separation alone. The GradSurgery experiment is recorded here as negative-result evidence: gradient-level projection does not reliably improve on architectural separation when the separation is already enforced by PLE’s expert routing.

Financial Operational Regimes and Task Coupling: Hypothesis vs. Current Evidence

While the observation of adaTT failing in this setting still stood, we developed an operational-regime explanation for why. At Meta’s *population-scale* data, task-to-task gradient relationships (CTR/CVR) are stable across billions of samples, so adaTT’s loss-level transfer reflects real structure. Financial institutions, with 10–20M customer *sample-scale* data, face distribution drift — rate hikes, product policy changes, regulatory shifts — that can invert task relationships, so loss-level coupling risks *correlated model failure* across linked tasks. Retraining cadence (hourly for big tech vs. weekly/monthly for finance) then determines whether that coupling cost is affordable.

The hypothesis is appealing and operationally useful. But the -0.019 AUC gap that anchored it collapsed to -0.001 after the five porting bugs were fixed. The narrative of “big-tech MTL methods being counterproductive in finance” is therefore not currently supported by empirical evidence on this dataset. What remains is a theoretical argument: in regimes with frequent distribution drift and limited sample size, loss-level task coupling is harder to lifecycle-manage than isolated architectures — still a legitimate design consideration for financial AI deployment.

The documentation policy is to keep this discussion rather than delete it. The distinction between population and sample regimes, and the concern about correlated failure modes, reflects real risks that inform deployment choices. What we should not do is treat this concern as empirically proven by adaTT’s own measured failure — it was not. It stays as an open hypothesis awaiting re-validation under genuine distribution-drift conditions (real bank data, post-rate-shock retraining) rather than as a confirmed finding.

Self-Regulating Experts: When Silence Is the Best Signal

The joint ablation analysis exposed a specific and unexpected culprit. Among the nine expert types, per-task metric decomposition showed that the Causal expert — implemented via NOTEARS — produced a -0.122 F1-macro drop on the `segment_prediction` task alone. No other single expert caused damage of that magnitude.

The investigation identified the structural reason. NOTEARS is an algorithm for recovering directed acyclic graphs (DAGs) from observational data. It always outputs a DAG — even when the data contains no genuine causal structure. In the synthetic dataset used for this project, labels are derived from formula-based transformations of features. NOTEARS finds statistically consistent edges in this setting, but those edges describe the formula, not real causal relationships. The algorithm learns spurious dependencies and encodes them as confident, dense representations. The Causal expert does not produce noise; it produces confidently wrong information at full signal strength.

The core problem is architectural: the Causal expert has no “I don’t know” option. Every expert in the current PLE design always produces an output vector, regardless of whether it has anything meaningful to contribute. When an expert’s output is confidently wrong, the PLE gate must learn to ignore it — but the gate’s softmax formulation means it can only suppress an expert by boosting others, not by eliminating the harmful signal entirely.

The solution is an internal confidence gate. If the total edge weight of the NOTEARS DAG falls below a learned threshold, the expert outputs a zero vector — silence — instead of a low-confidence representation. The PLE gate then receives nothing from that expert, rather than a misleading vector it must learn to discount.

The broader implication extends to every expert in the architecture. A self-regulating expert that silences itself when it cannot contribute useful signal eliminates the need for the PLE gate to identify which experts are harmful. New expert types can be added freely without the risk of negative transfer — each expert assumes responsibility for knowing when not to speak. The gate’s job simplifies from “which experts are harmful?” to “how should I weight experts that have already certified their own relevance?”

Self-Regulation as a Prerequisite for Expert Ensembling

This discovery connects directly to Paper 3’s central question: how to ensemble heterogeneous expert outputs whose semantics are incommensurable. Self-regulation is a prerequisite for principled ensembling. If some experts contribute confident noise, the ensemble signal is corrupted before any weighting scheme is applied. Self-regulating experts — those that output silence when their internal confidence is low — ensure that every vector the gate receives is a genuine claim of relevance, not a mandatory contribution. The gate can then ensemble on the basis of relative expertise rather than relative harmfulness.

Overfitting, Gate Entropy, and the Road to Adaptive Distillation

30-Epoch Overfitting: Loss-Metric Decoupling

The 30-epoch training run produced a finding that would have been easy to miss without per-task metric tracking. Training loss continued to decline monotonically after epoch 15, which is the expected behavior for a well-regularized model. Simultaneously, validation AUC on the classification tasks plateaued and then reversed, while regression MAE improved. Loss and metric were moving in opposite directions.

The decoupling has a structural cause specific to multi-task learning. The model was optimizing the uncertainty-weighted composite loss — a scalar that aggregates 13 per-task losses with learned s_k weights. As the s_k parameters adapted, the model effectively shifted weight away from classification tasks (where validation performance was degrading) toward regression tasks (where it continued to improve). The composite loss kept declining because the reweighting masked the classification deterioration. A single-task loss curve would have shown the plateau; the multi-task aggregate obscured it.

The prescribed response — cosine learning rate restarts — introduced a secondary problem. Each restart injected momentum into the optimization trajectory, which caused loss oscillation rather than smooth convergence. The restart amplitude was calibrated for single-task training budgets and was too large relative to the 13-task loss landscape. The oscillation was not harmful in isolation, but it made it impossible to identify a clean convergence epoch for checkpoint selection.

Lesson: Per-Task Metrics Are Non-Negotiable

Composite loss is useful for optimization but deceptive for monitoring. A model can achieve monotonically decreasing composite loss while silently degrading on a subset of tasks. Separate validation curves per task type — `avg_auc` for binary, `avg_f1_macro` for multiclass, `avg_mae` for regression — are the only reliable signal for multi-task convergence detection.

Gate Entropy: CGC Differentiates, Attention Does Not

A parallel diagnostic examined the distribution of gate weights across training runs. The CGC layer gates showed meaningful differentiation by epoch 10: values ranged from 0.33 (low reliance on a single expert) to 0.88 (near-exclusive reliance). This is the expected behavior — different tasks developing different expert preferences as the model discovers which inductive biases are useful for which prediction targets.

The per-task attention layer showed the opposite pattern. Attention weights converged to near-uniform distributions (entropy ≈ 1.0) and remained there throughout training. The attention mechanism was not learning to focus; it was averaging.

The diagnosis points to a capacity mismatch. The CGC gate operates over 7 heterogeneous expert outputs — semantically distinct vectors with different statistical properties. The gate has a strong training signal because expert outputs genuinely differ. The per-task attention operates over representations that, by the time they reach the attention layer, have been processed through two CGC stages and are more similar to each other. The attention gradient is weaker because the representations are less differentiated.

This finding informed the architecture for subsequent experiments: CGC gate weights are informative and can be used directly as explainability signals; per-task attention weights cannot be trusted as explanations at the current architecture scale.

3-Layer Fallback Architecture

Knowledge distillation from PLE teacher to LGBM student proved harder than anticipated. The first distillation attempt used standard soft-label transfer (temperature=5.0, alpha=0.3) and produced a student with adequate regression fidelity but poor classification alignment on the minority-class tasks.

The failure mode was diagnostic: the teacher’s soft labels for rare positive classes contained entropy that the student, with its lower capacity, could not absorb. The student learned the majority-class distribution well and treated the rare-class signal as noise.

Three failures of this type — on different task subsets — established that no single distillation configuration could satisfy all 13 tasks simultaneously. The resulting design is a 3-layer fallback architecture:

Layer	Model	When Used	Latency
L1	LGBM student (distilled)	Default: fidelity > threshold on all tasks	~5ms
L2a	PLE teacher (direct inference)	LGBM fidelity drops on binary/multiclass tasks	~80ms
L2b	Bedrock LLM (Sonnet)	Regulatory explanation required or confidence < floor	~800ms
L3	Rule-based fallback	All models unavailable, circuit breaker open	<1ms

The FallbackRouter monitors fidelity metrics on a rolling window. When LGBM fidelity on any task type falls below a configurable threshold, the router escalates to the next layer. The teacher (L2a) serves as a reliability backstop rather than the primary serving path, preserving the latency advantage of the student under normal conditions.

Adaptive Distillation: Teacher Threshold Gating and Floor Skip

Standard distillation transfers soft labels unconditionally. In a 13-task setup with significant class imbalance, this means the student receives low-confidence teacher outputs for rare classes and attempts to fit them — a form of teacher noise amplification.

Adaptive distillation adds two mechanisms. First, teacher threshold gating: soft labels from the PLE teacher are only transferred to the student when the teacher’s own confidence (measured by evidential uncertainty $u_k = \frac{K}{\sum(\alpha)}$ for multiclass tasks) exceeds a task-specific threshold. Below threshold, the student falls back to the hard label for that sample-task pair. Second, floor SKIP: if the teacher’s

confidence is below a minimum floor on a given task, the distillation loss for that task-sample pair is set to zero — the student is not penalized for disagreeing with an uncertain teacher.

The combined effect is that the student learns from teacher outputs that the teacher is confident about, and ignores task-sample pairs where the teacher is uncertain. This concentrates the distillation signal on the high-confidence, high-information-density portion of the teacher’s output distribution.

3-Agent Reason Pipeline: FactExtractor → TemplateEngine → SelfChecker

The recommendation rationale generation system was restructured from a single LLM call into a 3-agent pipeline, fully connected via Bedrock (Claude Sonnet):

1. **FactExtractor**: Receives the raw model outputs — gate weights, evidential uncertainty, IG attribution top features, contrastive pairs — and produces a structured fact bundle. No natural language generation at this stage; only structured extraction and validation.
2. **TemplateEngine**: Receives the fact bundle and produces a draft rationale in the appropriate register (customer-facing, advisor-facing, or regulator-facing). Template selection is driven by the request context, not by the LLM’s inference.
3. **SelfChecker**: Receives the draft rationale and the original fact bundle and verifies consistency. Checks that every claim in the rationale can be traced to a fact in the bundle. Returns either an approval or a specific inconsistency report that routes back to TemplateEngine.

The 3-agent structure solves a problem that emerged in single-LLM rationale generation: hallucination of model-internal details. A single LLM prompted with “explain this recommendation” would occasionally generate plausible but fabricated feature attributions. The FactExtractor→SelfChecker loop makes this impossible by grounding every rationale claim in verified model outputs. SageMaker end-to-end cost for the full serving pipeline including rationale generation runs at approximately \$0.69 per cycle under the current configuration.

Future Plans

Academic and Industry Publications

- **Paper Publication**: 2 papers completed (Paper 1: architecture + ablation, Paper 2: serving + ops/audit + compliance)
- **DuckDB Community**: Case study on replacing pandas with DuckDB as ML pipeline engine
- **Anthropic Case Study**: Building a financial AI system with Claude Code
- **GARP Submission**: Practitioner paper combining FRM credential with AI risk management
- **FSS Regulatory Review**: Compliance reference material for AI guidelines development

Regulatory Engagement

- **FSS AI Basic Act Compliance Review Request:** As enforcement decrees and guidelines for the AI Basic Act are being drafted, the team plans to request a review of this system’s explainability framework at the appropriate timing.

Follow-up Work

- **On-premises Production Data Results:** Performance results from actual production data to be added as paper supplements
- **Public GitHub Repository:** A sanitized version of the codebase with organizational information removed will be published as a public repository

*This project was completed not by overcoming a “lack of resources”
but by “redefining resources.”
It demonstrates that one desktop GPU combined with AI agents
can substitute for dedicated infrastructure.*

This document was written with the assistance of Claude Code (Anthropic). The human author directed the architectural decisions, experimental design, and interpretation of results; Claude Code assisted with code implementation, document drafting, and experimental execution.